ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

TOÁN ỨNG DỤNG VÀ THỐNG KÊ

BÁO CÁO ĐỒ ÁN 1
MA TRẬN VÀ CƠ SỞ CỦA TÍNH TOÁN KHOA HỌC

GVTH: ThS. VÕ NAM THỤC ĐOAN
ThS. LÊ NHỰT NAM

Lớp CQ2024/2
Nhóm 8

[Nguyễn Anh Thái]	[24120224]
[Nguyễn Đình Tuấn]	[24120237]
[Nguyễn Huỳnh Gia Bảo]	[24120264]
[Vòng Sau Hậu]	[24120307]
[Lương Nhật Tân]	[24120134]

Thành phố Hồ Chí Minh, ngày 26/4/2026.

Mục lục

1 Lời cảm ơn **4**

2 Giới thiệu thành viên **4**

3 Công cụ và môi trường phát triển **5**

4 Phần 1: Phép khử Gauss và Các Ứng Dụng **6**

A Phép khử Gauss và giải hệ phương trình tuyến tính 6

A.1 Mục tiêu và Bài toán 6

A.2 Cơ sở lý thuyết 6

A.3 Thuật toán 7

A.4 Mã giả 9

A.5 Cài đặt và Các hàm chính 10

A.6 Kết quả thực nghiệm (Test Cases) và Đánh giá 14

B Tính định thức của ma trận 17

B.1 Mục tiêu và Bài toán 17

B.2 Cơ sở lý thuyết 18

B.3 Thuật toán 18

B.4 Mã giả 20

B.5 Cài đặt và các hàm chính 20

B.6 Kết quả thực nghiệm (Test Cases) và Đánh giá 23

C Tìm ma trận nghịch đảo 26

C.1 Mục tiêu và Bài toán 26

C.2 Cơ sở lý thuyết 26

C.3 Thuật toán 26

C.4 Mã giả 27

C.5 Cài đặt và Các hàm chính 28

C.6 Kết quả thực nghiệm (Test Cases) và Đánh giá 30

D Tính hạng và cơ sở của các không gian 32

D.1 Mục tiêu và Bài toán 32

D.2 Cơ sở lý thuyết 32

D.3 Thuật toán 33

D.4 Mã giả 36

D.5 Cài đặt và Các hàm chính 38

D.6 Kết quả thực nghiệm (Test Cases) và Đánh giá 44

5 Phần 2: Phân rã ma trận và trực quan hóa với Manim 47
A Phân rã LU (LU Decomposition) 47
A.1 Cơ sở lý thuyết và Mục tiêu 47
A.2 Thuật toán 47
A.3 Mã giả 49
A.4 Cài đặt và Các hàm chính 49
A.5 Kết quả thực nghiệm (Test Cases) và Đánh giá 51
B Phân rã QR (QR Decomposition) 54
B.1 Cơ sở lý thuyết và Mục tiêu 54
B.2 Thuật toán 54
B.3 Mã giả 56
B.4 Cài đặt và Các hàm chính 56
B.5 Kết quả thực nghiệm (Test Cases) và Đánh giá 58
C Phân rã SVD (Singular Value Decomposition) 62
C.1 Cơ sở lý thuyết và Mục tiêu 62
C.2 Thuật toán 63
C.3 Mã giả 65
C.4 Cài đặt và các hàm chính 67
C.5 Kết quả thực nghiệm (Test Cases) và Đánh giá 71
D Chéo hóa ma trận 74
D.1 Cơ sở lý thuyết và Mục tiêu 74
D.2 Thuật toán 75
D.3 Mã giả 76
D.4 Cài đặt và các hàm chính 77
D.5 Kết quả thực nghiệm (Test Cases) và Đánh giá 81
E Trực quan hóa bằng Manim 84
E.1 Phân rã LU với kỹ thuật Partial Pivoting 84
E.2 Phân rã QR bằng phương pháp Gram-Schmidt 84
E.3 Phân rã SVD và Ý nghĩa hình học 85
E.4 Chéo hóa Ma trận $A = PDP^{-1}$ 85
E.5 Ứng dụng SVD: Nén ảnh (Image Compression) 86
E.6 Ứng dụng LU: Giải hệ phương trình tuyến tính 86
E.7 Truy cập video Trực quan hóa 87

6 Part 3: Giải hệ phương trình và Phân tích hiệu năng 88
A Lý thuyết sai số và Tính ổn định số 88
A.1 Số điều kiện (Condition Number) 88

A.2 So sánh các phương pháp giải hệ phương trình 88
A.3 Phương pháp lặp Gauss-Seidel 89
B Cài đặt và phân tích thực nghiệm 91
B.1 Ma trận ngẫu nhiên bình thường 92
B.2 Ma trận SPD (Well-conditioned) 95
B.3 Ma trận Hilbert (Ill-conditioned) 98
C Tổng kết 101

7 KẾT LUẬN 102
A Tóm tắt kết quả đạt được 102
B Bài học rút ra 102
C Khó khăn gặp phải 103

8 Tài liệu tham khảo 104

1 Lời cảm ơn

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên Võ Nam Thục Đoan và thầy Lê Nhựt Nam, bộ môn Toán Ứng Dụng và Thống Kê, Trường Đại học Khoa học Tự nhiên – ĐHQG TP.HCM, vì đã tận tình giảng dạy và hướng dẫn chúng em trong suốt quá trình học tập và thực hiện đồ án. Trong quá trình nghiên cứu và triển khai đề tài Ma Trận và Cơ Sở của Tính Toán Khoa Học, nhóm đã có cơ hội vận dụng kiến thức lý thuyết vào thực tế, đồng thời rèn luyện thêm kỹ năng làm việc nhóm và tư duy giải quyết vấn đề khi tập trung vào ba nhóm kỹ thuật cốt lõi của đại số tuyến tính trên máy tính.

Nhóm cũng xin chân thành cảm ơn các anh chị, bạn bè và gia đình đã hỗ trợ trong quá trình tìm kiếm tài liệu, tham gia thử nghiệm hệ thống và góp ý để bài làm được hoàn thiện hơn. Quá trình cùng nhau thực hiện đồ án giúp chúng em rèn luyện thêm kỹ năng tính toán, tinh thần hợp tác và ý thức trách nhiệm trong công việc nhóm. Đây là trải nghiệm quý báu, giúp chúng em hiểu sâu hơn về đại số tuyến tính và chuẩn bị tốt hơn cho hành trình học tập, nghiên cứu và làm việc sau này.

2 Giới thiệu thành viên

STT	Họ và tên	MSSV	Công việc	Đóng góp
1	Nguyễn Đình Tuấn	24120237	Giải hệ phương trình tuyến tính, phân rã LU, cài đặt và phân tích thực nghiệm	100%
2	Nguyễn Anh Thái	24120224	Hàm kiểm thử, ứng dụng phân rã, tổng hợp báo cáo, video và notebook	100%
3	Nguyễn Huỳnh Gia Bảo	24120264	Tìm ma trận nghịch đảo, phân rã SVD, cài đặt và phân tích thực nghiệm	100%
4	Vòng Sau Hậu	24120307	Tính hạng và cơ sở của các không gian, phân rã SVD, cài đặt và phân tích thực nghiệm	100%
5	Lương Nhật Tân	24120134	Tính định thức của ma trận, chéo hóa ma trận, cài đặt các phương pháp giải $Ax = b$	100%

3 Công cụ và môi trường phát triển

- **Python:** ngôn ngữ cài đặt chính, version 3.14.4 (phiên bản mới nhất).
- **NumPy, SciPy, SymPy:** kiểm chứng và so sánh kết quả (không dùng để cài đặt thuật toán).
- **Manim Community:** Trực quan hóa và tạo video demo, version 0.20.1 (phiên bản stable mới nhất).
- **Matplotlib:** Vẽ biểu đồ phân tích hiệu năng.
- **Jupyter Notebook:** Trình bày kết quả thực nghiệm.

4 Phần 1: Phép khử Gauss và Các Ứng Dụng

A Phép khử Gauss và giải hệ phương trình tuyến tính

A.1 Mục tiêu và Bài toán

Cho hệ phương trình tuyến tính $\mathbf{Ax} = \mathbf{b}$ với $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$. Nhiệm vụ của phần này là xây dựng một chương trình có khả năng:

- Đưa ma trận tăng cường $[\mathbf{A}|\mathbf{b}]$ về dạng bậc thang dòng (REF).
- Biện luận số lượng nghiệm (Duy nhất, Vô nghiệm, Vô số nghiệm).
- Đảm bảo tính ổn định số học thông qua kỹ thuật chọn phần tử chốt (Partial Pivoting).

A.2 Cơ sở lý thuyết

Xét hệ phương trình tuyến tính $\mathbf{Ax} = \mathbf{b}$ với $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$.

Định nghĩa A.2.1 (Ma trận tăng cường). *Ma trận tăng cường* (augmented matrix) của hệ $\mathbf{Ax} = \mathbf{b}$ là:

$$[\mathbf{A} \mid \mathbf{b}] = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix}$$

Định nghĩa A.2.2 (Các phép biến đổi dòng sơ cấp). Ba phép biến đổi dòng sơ cấp (elementary row operations) trên ma trận:

1. $R_i \leftrightarrow R_j$: Hoán đổi dòng i và dòng j.
2. $R_i \leftarrow c \cdot R_i$, với $c \neq 0$: Nhân dòng i với hằng số c.
3. $R_i \leftarrow R_i + c \cdot R_j$: Cộng c lần dòng j vào dòng i.

Ba phép biến đổi này **không làm thay đổi tập nghiệm** của hệ phương trình.

Định nghĩa A.2.3 (Dạng bậc thang dòng - REF). Ma trận **U** ở *dạng bậc thang dòng* (Row Echelon Form, viết tắt REF) nếu:

1. Mọi dòng toàn số 0 nằm phía dưới các dòng khác 0.
2. Phần tử chỉ huy (pivot) của mỗi dòng khác 0 nằm bên phải pivot của dòng ngay trên nó.

Định nghĩa A.2.4 (Khử Gauss có chọn phần tử chốt - Partial Pivoting).

Trong cài đặt thực tế, cần áp dụng *khử Gauss có chọn phần tử chốt* hay *partial pivoting* để đảm bảo tính ổn định số học.

Tại bước khử thứ k, chọn hàng p sao cho:

$$\left|a_{pk}^{(k)}\right| = \max_{k \le i \le n} \left|a_{ik}^{(k)}\right|, \tag{2}$$

Sau đó hoán đổi dòng k và dòng p trước khi thực hiện khử. Kỹ thuật này giảm thiểu đáng kể sai số làm tròn (round-off error) phát sinh trong tính toán dấu phẩy động.

A.3 Thuật toán

A.3.1. Kỹ thuật chọn phần tử chốt (Partial Pivoting):

Trong quá trình khử Gauss, nếu phần tử chốt M_{kk} có giá trị tuyệt đối quá nhỏ, các phép chia $\frac{M_{ik}}{M_{kk}}$ sẽ khuếch đại sai số làm tròn, dẫn đến hiện tượng mất ổn định số học (Ill-conditioned).

Kỹ thuật của nhóm thực hiện tìm:

$$p = \underset{i \in [k, n-1]}{\operatorname{argmax}} |M_{ik}|$$

Sau đó, hoán đổi dòng p với dòng k hiện tại.

Nếu $|M_{pk}| < \varepsilon$, hệ thống sẽ đưa ra cảnh báo về độ tin cậy của nghiệm.

A.3.2. Biện luận nghiệm tổng quát:

Dựa trên ma trận sau khi khử Gauss, hạng của ma trận hệ số $\text{rank}(A)$ và hạng của ma trận tăng cường $\text{rank}(M)$ được sử dụng để kết luận:

- **Vô nghiệm:** Nếu tồn tại dòng có dạng $[0, 0, \ldots, 0 \mid b'_i]$ với $b'_i \neq 0$.
- **Nghiệm duy nhất:** Nếu không vô nghiệm và không có ẩn tự do.
- **Vô số nghiệm:** Nếu không vô nghiệm và có ẩn tự do (các ẩn không tương ứng với cột pivot).

A.4 Mã giả

Algorithm 1 Khử Gauss với Chọn phần tử chốt một phần

Input: Ma trận hệ số $A \in \mathbb{R}^{n \times m}$, vector vế phải $b \in \mathbb{R}^n$
Output: Ma trận tăng cường M ở dạng bậc thang, số lần đổi dòng s
Khởi tạo ma trận tăng cường $M \leftarrow [A|b]$
$pivot_row \leftarrow 0, \quad pivot_col \leftarrow 0, \quad s \leftarrow 0$
while $pivot_row < n$ **and** $pivot_col < m$ **do**
    Tìm hàng $p \in [pivot_row, n-1]$ sao cho $|M_{p,pivot_col}|$ đạt giá trị lớn nhất
    $max_val \leftarrow |M_{p,pivot_col}|$
    if $max_val = 0$ **then** ▷ Không tìm thấy pivot tại cột này
        $pivot_col \leftarrow pivot_col + 1$
        continue
    end if
    if $max_val < \varepsilon$ **then** ▷ Cảnh báo mất ổn định số học
        Cảnh báo: "Hệ có thể ill-conditioned"
    end if
    if $p \neq pivot_row$ **then** ▷ Hoán đổi dòng
        Hoán đổi dòng M_p và M_{pivot_row}
        $s \leftarrow s + 1$
    end if
    for $i = pivot_row + 1$ **to** $n - 1$ **do** ▷ Khử các hàng bên dưới
        $l \leftarrow M_{i,pivot_col} / M_{pivot_row,pivot_col}$
        $M_{i,pivot_col} \leftarrow 0$
        for $j = pivot_col + 1$ **to** m **do**
            $M_{i,j} \leftarrow M_{i,j} - l \times M_{pivot_row,j}$
        end for
    end for
    $pivot_row \leftarrow pivot_row + 1$
    $pivot_col \leftarrow pivot_col + 1$
end while
$U \leftarrow$ Ma trận M trừ cột cuối
$c \leftarrow$ Cột cuối ma trận M
$x \leftarrow$ Thế ngược(U, c)
return M, x, s

Algorithm 2 Thế ngược và Biện luận nghiệm

Input: Ma trận bậc thang $U \in \mathbb{R}^{n \times m}$, vector vế phải $c \in \mathbb{R}^n$
Output: Danh sách nghiệm x (chứa giá trị số hoặc biểu thức ẩn tự do)
Xác định các vị trí pivot (r_k, c_k) trong U. Gọi *rank* là số lượng pivot.
for $i = rank$ **to** $n-1$ **do** ▷ Kiểm tra tính tương thích
    if $|c_i| \neq 0$ **then**
        return "Hệ phương trình Vô nghiệm"
    end if
end for
Xác định tập hợp ẩn tự do *FreeVars* (các cột không chứa pivot)
Khởi tạo $x_j =$ "x$_j$" với mọi $j \in FreeVars$
for mỗi vị trí pivot (i, j) duyệt ngược từ cuối lên đầu **do**
    $const_part \leftarrow c_i$
    $expr_part \leftarrow$ Danh sách rỗng
    for $k = j+1$ **to** $m-1$ **do**
        if x_k là giá trị số **then**
            $const_part \leftarrow const_part - U_{i,k} \times x_k$
        else ▷ Trường hợp ẩn k là ẩn tự do hoặc biểu thức
            Thêm biểu thức phụ thuộc ẩn tự do vào $expr_part$
        end if
    end for
    if $expr_part$ trống **then**
        $x_j \leftarrow const_part / U_{i,j}$ ▷ Nghiệm số cụ thể
    else
        $x_j \leftarrow$ Biểu thức tổng quát theo các ẩn tự do
    end if
end for
return x

A.5 Cài đặt và Các hàm chính

A.5.1. Hàm khử Gauss với kỹ thuật chọn phần tử chốt (Partial Pivoting)

```python
def gaussian_eliminate(A, b):
    # Tao ma tran mo rong M = [A|b]
    M = []
    num_rows = len(A)
    num_cols = len(A[0])

    for i in range(num_rows):
        M.append(A[i] + [b[i]])

    s = 0
    epsilon = 1e-12
```

```
    pivot_row = 0
    pivot_col = 0

    while pivot_row < num_rows and pivot_col < num_cols:
        p = pivot_row
        max_val = abs(M[pivot_row][pivot_col])

        # Tim phan tu chot lon nhat (Partial Pivoting)
        for i in range(pivot_row + 1, num_rows):
            if abs(M[i][pivot_col]) > max_val:
                max_val = abs(M[i][pivot_col])
                p = i

        if max_val == 0:
            pivot_col += 1
            continue

        if max_val < epsilon:
            warnings.warn(f"Pivot tai cot {pivot_col} gan bang 0 ({max_val:.2e}). He co the ill-conditioned.")

        # Hoan doi dong
        if p != pivot_row:
            M[p], M[pivot_row] = M[pivot_row], M[p]
            s += 1

        # Khu Gauss
        for i in range(pivot_row + 1, num_rows):
            l = M[i][pivot_col] / M[pivot_row][pivot_col]
            for j in range(pivot_col, num_cols + 1):
                M[i][j] -= l * M[pivot_row][j]

        pivot_row += 1
        pivot_col += 1

    # Tach ma tran U va vector c de the nguoc
    U = [row[:num_cols] for row in M]
    c = [row[num_cols] for row in M]
    x = back_substitution(U, c)

    return M, x, s
```

- **Độ phức tạp thời gian:** $O(n^2m)$ (hoặc $O(n^3)$ đối với ma trận vuông). Chi phí này

đến từ vòng lặp `while` duyệt qua các đường chéo, kết hợp với 2 vòng lặp `for` lồng nhau để cập nhật các phần tử trong bước triệt tiêu dòng.

- **Độ phức tạp không gian:** $O(n \cdot m)$ để khởi tạo và lưu trữ ma trận mở rộng $M = [A|b]$ cùng với ma trận U tách ra ở cuối.
- **Tính ổn định:** Thuật toán đạt độ ổn định số học cao nhờ kỹ thuật Partial Pivoting. Đồng thời, việc sử dụng hằng số $\varepsilon = 10^{-12}$ giúp phát hiện và cảnh báo sớm các ma trận có nguy cơ bị điều kiện kém (ill-conditioned).

A.5.2. Hàm thế ngược để biện luận nghiệm.

```python
def back_substitution(U, c):
    num_rows = len(U)
    num_cols = len(U[0])
    pivot_indices = []

    # Xac dinh vi tri cac pivot
    for row in range(num_rows):
        has_pivot = False
        for col in range(num_cols):
            if abs(U[row][col]) != 0:
                pivot_indices.append((row, col))
                has_pivot = True
                break
        if not has_pivot and abs(c[row]) != 0:
            return 'Vo nghiem'

    pivot_cols = [p[1] for p in pivot_indices]
    free_cols = [i for i in range(num_cols) if i not in pivot_cols
    ]

    x = [None] * num_cols
    for j in range(num_cols):
        if j not in pivot_cols:
            x[j] = f'x{j+1}'

    # The nguoc tu duoi len
    for i, j in reversed(pivot_indices):
        num_val = c[i]
        not_num_val = []
        for k in range(j + 1, num_cols):
            if abs(U[i][k]) != 0:
                if k in free_cols:
```

```
                    sign = '-' if U[i][k] > 0 else '+'
                    not_num_val.append(f'{sign}{abs(U[i][k])}*x{k+1}')
                elif isinstance(x[k], (int, float)):
                    num_val -= U[i][k] * x[k]

        if len(not_num_val) == 0:
            x[j] = num_val / U[i][j]
        else:
            res = f'{num_val}/{U[i][j]}'
            for item in not_num_val:
                if item[0] == '+':
                    res += f' + {item[1:]}'
                else:
                    res += f' - {item[1:]}'
            res = '(' + res + ')' + f'/{U[i][j]}'
            x[j] = res

    return x
```

- **Độ phức tạp thời gian:** $O(n \cdot m)$ (hoặc $O(n^2)$ với hệ vuông). Quá trình quét tìm các vị trí chốt (pivots) mất $O(n \cdot m)$, và quá trình tính toán thế ngược từ dưới lên cũng chỉ duyệt qua ma trận một lần.
- **Độ phức tạp không gian:** $O(m)$ (độ phức tạp bộ nhớ phụ) để lưu trữ danh sách các cột chốt, cột tự do và mảng lưu trữ nghiệm x.
- **Tính linh hoạt:** Hàm xử lý tốt các tình huống biên bằng cách trả về chuỗi biện luận biểu thức cho các ẩn tự do, cũng như phát hiện chính xác trường hợp vô nghiệm khi tồn tại dòng $[0|c]$ với $c \neq 0$.

A.5.3. Hàm kiểm thử kết quả Gauss bằng NumPy

```
def verify_solution(A, x, b):

    A_np = np.array(A, dtype=float)
    b_np = np.array(b, dtype=float)
    x_custom = np.array(x, dtype=float)

    try:
        x_numpy = np.linalg.solve(A_np, b_np)
        is_match = np.allclose(x_custom, x_numpy)
```

```python

        b_calculated = np.dot(A_np, x_custom)
        is_b_match = np.allclose(b_calculated, b_np)

        if is_match and is_b_match:
            print("Ket qua cua ban la dung.")
            return True
        else:
            print("Ket qua cua ban la sai.")
            print(f"Ket qua NumPy: {x_custom}")
            return False

    except np.linalg.LinAlgError as e:
        print(f"Loi NumPy: {e}")
```

A.6 Kết quả thực nghiệm (Test Cases) và Đánh giá

Kịch bản A.6.1: Hệ phương trình có nghiệm duy nhất. Xét hệ phương trình với ma trận hệ số không suy biến:

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 1 & 1 \\ -1 & 1 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 8 \\ 5 \end{bmatrix}$$

- Ma trận sau khi khử: $\begin{bmatrix} 2 & 2 & 1 & 8 \\ 0.0 & 1.5 & -1.5 & -3.0 \\ 0.0 & 0.0 & 2.0 & 12.0 \end{bmatrix}$
- Nghiệm thu được: $x = [-1.0, 4.0, 6.0]^T$.
- Số lần hoán đổi dòng (s): 1.
- Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

Kịch bản A.6.2: Hệ phương trình vô nghiệm. Kiểm tra khả năng phát hiện mâu thuẫn sau khi khử Gauss:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 5 \\ 3 \end{bmatrix}$$

- Ma trận sau khi khử: $\begin{bmatrix} 2 & 2 & 2 & 5 \\ 0.0 & 1.0 & 2.0 & 0.5 \\ 0.0 & 0.0 & 0.0 & -1.5 \end{bmatrix}$
- Nghiệm thu được: Vô nghiệm
- Số lần hoán đổi dòng (s): 2.
- Kết quả kiểm thử:

```
Loi NumPy: Singular matrix
```

Kịch bản A.6.3: Hệ phương trình vô số nghiệm (Có ẩn cố định). Đây là trường hợp quan trọng để kiểm tra logic xử lý ẩn tự do:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

- Ma trận sau khi khử: $\begin{bmatrix} 1 & 1 & 1 & 2 \\ 0.0 & 0.0 & 1.0 & 1.0 \end{bmatrix}$
- Nghiệm tổng quát: $x_3 = 1.0, \quad x_2 = \text{tự do}, \quad x_1 = 1.0 - 1.0x_2$.
- Số lần hoán đổi dòng (s): 0.
- Kết quả kiểm thử:

```
Loi NumPy: Last 2 dimemsions of the array must be square
```

Kịch bản A.6.4: Ma trận hình chữ nhật ($n < m$). Đây là trường hợp kiểm tra tính ổn định và chính xác khi làm việc với ma trận hình chữ nhật:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 8 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

- Ma trận sau khi khử: $\begin{bmatrix} 2 & 4 & 8 & 10 & 2 \\ 0.0 & 0.0 & -1.0 & -1.0 & 0.0 \end{bmatrix}$
- Nghiệm tổng quát: $x_1 = \frac{1-4x_2-10x_4}{2}, \quad x_2 = \text{tự do}, \quad x_3 = \frac{1.0x_4}{-1.0}, \quad x_4 = \text{tự do}$.
- Số lần hoán đổi dòng (s): 1.
- Kết quả kiểm thử:

```
Loi NumPy: Last 2 dimemsions of the array must be square
```

Kịch bản A.6.5: Ma trận cần Partial Pivoting (Phần tử chốt bằng 0). Kiểm tra tính ổn định số học khi hàng đầu tiên có pivot bằng 0:

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & -2 & -3 \\ 5 & -1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ 0 \\ -3 \end{bmatrix}$$

- Ma trận sau khi khử: $\begin{bmatrix} 5 & -1 & 2 & -3 \\ 0.0 & 2.0 & 1.0 & 4.0 \end{bmatrix}$
- Nghiệm thu được: $x = [0.64, 2.84, -1.6800000000000002]^T$.
- Số lần hoán đổi dòng (s): 2.
- Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

Kịch bản A.6.6: Ma trận có pivot nhỏ. Đây là trường hợp kiểm tra tính ổn định và chính xác khi làm việc với ma trận có pivot nhỏ

$$A = \begin{bmatrix} 0.0000000000001 & 1 \\ 0.0000000000002 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

- Ma trận sau khi khử: $\begin{bmatrix} 2e-13 & 1 & 2 \\ 0.0 & 0.5 & 0.0 \end{bmatrix}$
- Nghiệm thu được: $x = [10000000000000.0, 0.0]^T$.
- Số lần hoán đổi dòng (s): 1.
- Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

- Chương trình cảnh báo:

```
Pivot tai cot 0 gan bang 0 (2.00e-13). He co the ill-conditioned
```

A.6.7. Đánh giá

Kết quả thực nghiệm cho thấy thuật toán khử Gauss tích hợp thế ngược không chỉ giải chính xác các hệ phương trình tiêu chuẩn mà còn xử lý an toàn mọi tình huống biên phức tạp. Thuật toán phân loại và biện luận thành công các hệ vô nghiệm, hệ vô số nghiệm (biểu diễn đúng ẩn phụ thuộc theo ẩn tự do), và đặc biệt là hệ chữ nhật. Kỹ thuật Partial Pivoting tiếp tục chứng minh vai trò then chốt trong việc bảo vệ tính ổn định số học, giúp chương trình tránh được các sai số khuếch đại khi hệ số chốt quá nhỏ, đảm bảo nghiệm xuất ra đáng tin cậy trong môi trường tính toán thực tế.

B Tính định thức của ma trận

B.1 Mục tiêu và Bài toán

Cho một ma trận vuông $A \in \mathbb{R}^{n\times n}$. Định thức $\det(A)$ là một đại lượng vô hướng mang nhiều ý nghĩa quan trọng trong đại số tuyến tính:

- Cho biết ma trận có khả nghịch hay không
- Phản ánh hệ số co giãn thể tích của phép biến đổi tuyến tính tương ứng
- Xuất hiện trong công thức nghiệm của hệ phương trình tuyến tính (quy tắc Cramer)

Phương pháp tính định thức hiệu quả nhất trên máy tính là dựa vào **khử Gauss**: đưa ma trận A về dạng tam giác trên U, sau đó định thức bằng tích các phần tử trên đường chéo chính của U. Tuy nhiên:

- Nếu phần tử chốt (pivot) tại một bước nào đó bằng 0, quá trình khử sẽ thất bại do phép chia cho 0
- Pivot quá nhỏ gây ra sai số làm tròn nghiêm trọng trên máy tính dấu phẩy động.

Để khắc phục, thuật toán áp dụng kỹ thuật **chọn phần tử trội từng phần (Partial Pivoting)**: ở mỗi bước i, tìm hàng có giá trị tuyệt đối lớn nhất tại cột i rồi hoán vị hàng đó lên vị trí i. Mỗi lần hoán vị đổi dấu định thức một lần, do đó cần theo dõi số lần hoán vị k và điều chỉnh dấu cuối cùng theo hệ số $(-1)^k$. Công thức tổng quát là:

$$\det(A) = (-1)^k \prod_{i=1}^{n} u_{i,i}$$

Trong đó $u_{i,i}$ là các phần tử trên đường chéo chính của ma trận tam giác trên U sau khi khử Gauss, và k là tổng số lần hoán vị hàng đã thực hiện.

B.2 Cơ sở lý thuyết

- **Tính chất hoán vị hàng:** Hoán đổi hai hàng bất kỳ của ma trận sẽ đổi dấu định thức:
$$\det(\ldots, R_i, \ldots, R_j, \ldots) = -\det(\ldots, R_j, \ldots, R_i, \ldots)$$
Vì vậy, sau k lần hoán vị, định thức bị nhân thêm hệ số $(-1)^k$.

- **Bất biến qua phép khử sơ cấp:** Phép cộng bội một hàng vào hàng khác ($R_j \leftarrow R_j - \text{factor} \cdot R_i$) không làm thay đổi giá trị định thức. Đây chính là phép biến đổi sơ cấp được thực hiện trong bước khử Gauss.

- **Định thức ma trận tam giác:** Định thức của ma trận tam giác (trên hoặc dưới) bằng tích các phần tử trên đường chéo chính:
$$\det(U) = \prod_{i=1}^{n} u_{i,i}$$
Nếu bất kỳ $u_{i,i} = 0$, ma trận suy biến và $\det(A) = 0$.

B.3 Thuật toán

Quá trình tính định thức được thực hiện qua n bước lặp. Giả sử ta đang ở bước thứ i:

Bước B.3.1. Tìm phần tử trội (Partial Pivoting) Tìm chỉ số hàng p (với $i \leq p \leq n$) sao cho phần tử ở cột i đạt giá trị tuyệt đối lớn nhất:

$$|a_{p,i}| = \max_{i \leq j \leq n} |a_{j,i}|$$

Nếu $|a_{p,i}| < \varepsilon$ (với $\varepsilon = 10^{-12}$), ma trận suy biến tại cột này và thuật toán trả về $\det(A) = 0$ ngay lập tức.

Bước B.3.2: Hoán vị hàng và đếm số lần hoán vị Nếu $p \neq i$, hoán đổi hàng i và hàng p của ma trận làm việc:
$$\text{mat}[i] \leftrightarrow \text{mat}[p]$$

Tăng bộ đếm hoán vị: $k \leftarrow k+1$.

Bước B.3.3: Khử Gauss tiến Với mỗi hàng j từ $i+1$ đến n:

1. Tính hệ số triệt tiêu: $\text{factor} = \dfrac{a_{j,i}}{a_{i,i}}$.
2. Cập nhật các phần tử trên hàng j: $a_{j,k} \leftarrow a_{j,k} - \text{factor} \cdot a_{i,k} \quad (\forall k \geq i)$.

Sau bước này, tất cả các phần tử bên dưới vị trí (i,i) tại cột i đều bằng 0.

Bước B.3.4: Tính định thức cuối cùng Sau khi hoàn tất n bước khử, ma trận đã ở dạng tam giác trên. Tính tích các phần tử trên đường chéo chính, rồi điều chỉnh dấu:

$$\det(A) = (-1)^k \cdot \prod_{i=1}^{n} \text{mat}[i][i]$$

B.4 Mã giả

Algorithm 3 Tính định thức $\det(A)$ bằng khử Gauss với Partial Pivoting

Require: Ma trận vuông $A \in \mathbb{R}^{n \times n}$
Ensure: Giá trị định thức $\det(A) \in \mathbb{R}$

```
mat ← bản sao của A
k ← 0                                          ▷ Bộ đếm số lần hoán vị
for i = 0 to n − 1 do
    // Bước 1: Tìm phần tử trội
    pivot_val ← mat[i][i], pivot_row ← i
    for j = i + 1 to n − 1 do
        if |mat[j][i]| > |pivot_val| then
            pivot_val ← mat[j][i], pivot_row ← j
        end if
    end for
    if |pivot_val| < 10^−12 then
        return 0                               ▷ Ma trận suy biến
    end if
    // Bước 2: Hoán vị hàng
    if pivot_row ≠ i then
        Hoán đổi mat[i] và mat[pivot_row]
        k ← k + 1
    end if
    // Bước 3: Khử Gauss
    for j = i + 1 to n − 1 do
        factor ← mat[j][i] / pivot_val
        for l = i to n − 1 do
            mat[j][l] ← mat[j][l] − factor × mat[i][l]
        end for
    end for
end for
// Bước 4: Tích đường chéo và điều chỉnh dấu
det ← ∏_{i=0}^{n−1} mat[i][i]
if k mod 2 = 1 then
    det ← −det
end if
return det
```

B.5 Cài đặt và các hàm chính

B.5.1. Hàm tính định thực bằng khử Gauss với Partial Pivoting

```python
def copy_A(A):
```

```
    """Tao mot ban sao cua ma tran de tranh thay doi du lieu goc"""
    return [row[:] for row in A]

def determinant(A):
    n = len(A)
    # Sao chep ma tran lam viec de khong anh huong den ma tran truyen vao
    mat = copy_A(A)
    swaps_count = 0  # Bien dem so lan hoan vi dong (de doi dau dinh thuc)

    for i in range(n):
        # Mac dinh phan tu chot (pivot) la phan tu tren duong cheo chinh
        pivot_val = mat[i][i]
        pivot_row = i

        # Chien luoc chon phan tu chot mot phan (Partial Pivoting)
        # Tim dong co gia tri tuyet doi lon nhat o cot hien tai de giam sai so
        for j in range(i + 1, n):
            if abs(mat[j][i]) > abs(pivot_val):
                pivot_val = mat[j][i]
                pivot_row = j

        # Neu phan tu chot lon nhat van xap xi 0, ma tran suy bien (dinh thuc = 0)
        if abs(pivot_val) < 1e-12:
            return 0

        # Hoan vi dong hien tai voi dong chua phan tu chot lon nhat
        if pivot_row != i:
            mat[i], mat[pivot_row] = mat[pivot_row], mat[i]
            swaps_count += 1

        # Qua trinh khu Gauss de dua ma tran ve dang tam giac tren
        for j in range(i + 1, n):
            # Tinh he so triet tieu
            factor = mat[j][i] / pivot_val

            # Cap nhat cac phan tu con lai tren dong j
            for k in range(i, n):
```

```
                mat[j][k] -= factor * mat[i][k]

    # Dinh thuc cua ma tran tam giac bang tich cac phan tu tren duong cheo chinh
    det = 1.0
    for i in range(n):
        det *= mat[i][i]

    # Dieu chinh dau cua dinh thuc dua tren so lan hoan vi dong ((-1)^k)
    if swaps_count % 2 == 1:
        det = -det

    return det
```

- **Độ phức tạp thời gian:** $O(n^3)$ do áp dụng phương pháp biến đổi về dạng tam giác trên bằng khử Gauss (3 vòng lặp lồng nhau).
- **Độ phức tạp không gian:** $O(n^2)$ do cần tạo một bản sao độc lập của ma trận (`copy_A(A)`) để thực hiện các phép biến đổi dòng, giúp không làm thay đổi (mutate) ma trận gốc đầu vào.
- **Tính ổn định:** Việc sử dụng Partial Pivoting giúp hạn chế tối đa sai số khuếch đại. Ngoài ra, thuật toán được tối ưu để dừng sớm (early exit) và trả về 0 ngay lập tức nếu phát hiện ma trận suy biến (pivot ≈ 0).

B.5.2. Hàm kiểm thử định thức

```
def verify_determinant_solution(A, x):
    A_np = np.array(A, dtype=float)

    try:
        x_np = np.linalg.det(A_np)

        if np.isclose(x, x_np):
            print("Dinh thuc cua ban la dung.")
            return True
        else:
            print("Dinh thuc cua ban la sai.")
            return False
    except np.linalg.LinAlgError as e:
        print(f"Loi NumPy: {e}")
```

B.6 Kết quả thực nghiệm (Test Cases) và Đánh giá

Thuật toán tính định thức đã được cài đặt bằng Python và kết quả được đối chiếu với hàm `numpy.linalg.det` của thư viện `NumPy`. Các kịch bản kiểm thử được thiết kế để bao quát đầy đủ các dạng ma trận đặc trưng, nhằm kiểm chứng cả tính đúng đắn lẫn độ bền bỉ của thuật toán.

B.6.1. Ma trận vuông tiêu chuẩn (Baseline Case)

Đây là kịch bản cơ bản nhất để xác nhận thuật toán hoạt động đúng trên ma trận tổng quát, không có tính chất đặc biệt.

- **Độ chính xác:** Kết quả tính được trùng khớp hoàn toàn với `numpy.linalg.det`.
- **Xử lý Partial Pivoting:** Phép hoán vị được thực hiện và dấu định thức được điều chỉnh đúng theo số lần hoán vị.
- **Test Case 1:** Ma trận vuông (3×3) tổng quát.

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix}$$

Quá trình khử: hoán vị hàng 0 và hàng 1 (do $|4| > |2|$), $k = 1$. Sau khi khử Gauss:

$$U = \begin{bmatrix} 4 & -6 & 0 \\ 0 & 4 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \Rightarrow \quad \det(A) = (-1)^1 \times (4 \times 4 \times 1) = -16$$

Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

B.6.2. Ma trận suy biến (Singular Matrix)

Ma trận suy biến là ma trận có $\det(A) = 0$, tương đương với các hàng (hoặc cột) phụ thuộc tuyến tính. Thuật toán phát hiện trường hợp này khi phần tử trội tại một cột nào đó xấp xỉ 0, và lập tức trả về 0 mà không chia cho 0.

- **Test Case 2:** Ma trận suy biến do hàng phụ thuộc tuyến tính.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$$

Hàng 2 bằng đúng $2\times$ hàng 1, nên các hàng phụ thuộc tuyến tính. Sau bước khử đầu tiên, cột 0 của hàng 1 bị triệt tiêu hoàn toàn, và cột 1 của hàng 1 cũng về 0. Phần tử trội tại bước tiếp theo $= 0$:

$$\det(A) = 0$$

Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

- **Test Case 3:** Ma trận suy biến do cột toàn số 0.

$$A = \begin{bmatrix} 0 & 2 & 3 \\ 0 & 4 & 6 \\ 0 & 6 & 9 \end{bmatrix}$$

Cột đầu tiên toàn số 0, phần tử trội $= 0$ ngay tại bước đầu tiên. Thuật toán trả về $\det(A) = 0$ tức thì.

Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

B.6.3. Các trường hợp đặc biệt (Edge Cases)

Thuật toán thể hiện tính bền bỉ khi đối mặt với các ma trận có cấu trúc cực đoan:

- **Ma trận đơn vị:** Không có phép hoán vị nào được thực hiện ($k = 0$), mọi phần tử đường chéo bằng 1. Định thức trả về đúng bằng 1.

  Test Case 4: Ma trận đơn vị (3×3).

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \Rightarrow \quad \det(A) = (-1)^0 \times (1 \times 1 \times 1) = 1$$

Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

- **Ma trận tam giác trên:** Không cần thực hiện bất kỳ phép hoán vị hay khử nào vì ma trận đã ở dạng tối ưu. Định thức được tính ngay bằng tích đường chéo.

  Test Case 5: Ma trận tam giác trên (3×3)

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & -1 \\ 0 & 0 & 7 \end{bmatrix} \quad \Rightarrow \quad \det(A) = (-1)^0 \times (2 \times 5 \times 7) = 70$$

  Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

- **Ma trận đường chéo phụ (Anti-diagonal):** Thuật toán phải thực hiện nhiều lần hoán vị liên tiếp. Số lần hoán vị quyết định dấu của định thức.

  Test Case 6: Ma trận đường chéo phụ (3×3)

$$A = \begin{bmatrix} 0 & 0 & 3 \\ 0 & 2 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

  Bước $i = 0$: pivot_row $= 2$ (do $|1|$ là lớn nhất cột 0), hoán vị hàng 0 và 2, $k = 1$.
  Bước $i = 1$: pivot_row $= 1$ (đã là lớn nhất), không hoán vị.
  Sau khử Gauss: $U = \text{diag}(1, 2, 3)$, tích đường chéo $= 6$.

$$\det(A) = (-1)^1 \times 6 = -6$$

  Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

- **Ma trận** 1×1**:** Trường hợp suy biến của vòng lặp, thuật toán tính đúng ngay tại bước đầu tiên và trả về phần tử duy nhất làm định thức.

  Test Case 7: Ma trận 1×1

$$A = \begin{bmatrix} -5 \end{bmatrix} \quad \Rightarrow \quad \det(A) = -5$$

  Kết quả kiểm thử:

```
Dinh thuc cua ban la dung.
```

B.6.4. Đánh giá

Kết quả thực nghiệm cho thấy thuật toán không chỉ đảm bảo độ chính xác toán học tuyệt đối mà còn xử lý an toàn mọi tình huống biên, từ ma trận suy biến, ma trận có cấu trúc đặc biệt, đến ma trận đòi hỏi nhiều lần hoán vị liên tiếp. Kỹ thuật Partial Pivoting đóng vai trò then chốt trong việc đảm bảo tính ổn định số của thuật toán trong thực hành.

C Tìm ma trận nghịch đảo

C.1 Mục tiêu và Bài toán

Thuật toán Gauss-Jordan là một phương pháp đại số tuyến tính dùng để tìm ma trận nghịch đảo A^{-1} của một ma trận vuông A. Dựa trên lý thuyết về ma trận khả nghịch, ta biết rằng một ma trận A khả nghịch khi và chỉ khi nó tương đương dòng với ma trận đơn vị I_n. Quá trình này được thực hiện bằng cách thực hiện các phép biến đổi sơ cấp trên dòng để đưa ma trận mở rộng về dạng rút gọn.

C.2 Cơ sở lý thuyết

Điều kiện khả nghịch: *Ma trận* $\mathbf{A} \in \mathbb{R}^{n\times n}$ *khả nghịch khi và chỉ khi* $\det(\mathbf{A}) \neq 0$.

Phương pháp Gauss - Jordan: *Nghịch đảo* $\mathbf{A}^{-1}$ *được tìm bằng cách thực hiện biến đổi dòng đồng thời trên ma trận ghép* $[\mathbf{A} \mid \mathbf{I}_n]$ *cho đến khi đạt dạng* $[\mathbf{I}_n \mid \mathbf{A}^{-1}]$.

C.3 Thuật toán

Dựa trên phương pháp tìm ma trận nghịch đảo, ta thiết lập ma trận mở rộng $(A|I_n)$ và dùng các phép biến đổi sơ cấp dòng biến A về dạng ma trận bậc thang rút gọn:

$$(A|I_n) \xrightarrow{\varphi_1} (A_1|B_1) \xrightarrow{\varphi_2} \dots \xrightarrow{\varphi_p} (A_p|B_p) \to \dots$$

Trong quá trình biến đổi, thuật toán xử lý dựa trên hai trường hợp:

- **Trường hợp 1 (TH1):** Tồn tại bước biến đổi p sao cho ma trận A_p có ít nhất một dòng hoặc một cột bằng 0. Khi đó hệ số chốt $|M[i,i]|$ sẽ vô cùng nhỏ (xấp xỉ bằng

0). Thuật toán kết luận A không khả nghịch.

- **Trường hợp 2 (TH2):** Mọi ma trận A_i đều không có dòng hay cột bằng 0. Ta đưa được vế trái về ma trận đơn vị I_n. Khi đó, ma trận cuối cùng có dạng $(I_n|B)$, suy ra A khả nghịch và $A^{-1} = B$.

C.4 Mã giả

Algorithm 4 Thuật toán Gauss-Jordan tìm ma trận nghịch đảo

Require: Ma trận vuông A cấp n
Ensure: Ma trận nghịch đảo A^{-1} hoặc thông báo không khả nghịch

```
M ← [A | I_n] cấp n × 2n                                  ▷ Ép kiểu số thực (float)
for i = 0 to n − 1 do
    // Bước 1: Tìm phần tử trội (Pivoting)
    max_row ← i
    for k = i + 1 to n − 1 do
        if |M[k, i]| > |M[max_row, i]| then
            max_row ← k
        end if
    end for
    Hoán đổi dòng i và dòng max_row trong M
    if |M[i, i]| < 10^−10 then
        return "Ma trận không khả nghịch (TH1)"
    end if
    // Bước 2: Chuẩn hóa pivot về 1
    pivot ← M[i, i]
    for j = 0 to 2n − 1 do
        M[i, j] ← M[i, j]/pivot
    end for
    // Bước 3: Khử các dòng khác (Elimination)
    for k = 0 to n − 1 do
        if k ≠ i then
            factor ← M[k, i]
            for j = 0 to 2n − 1 do
                M[k, j] ← M[k, j] − factor × M[i, j]
            end for
        end if
    end for
end for
return Nửa bên phải (cột n đến 2n − 1) của ma trận M                    ▷ TH2
```

C.5 Cài đặt và Các hàm chính

C.5.1. Hàm tìm ma trận nghịch đảo bằng thuật toán Gauss-Jordan

```
def inverse(A):
    n = len(A)
    matrix = [] # Khoi tao ma tran

    # Tao ma tran (A|I)
    for i in range(n):
        row = []
        # Ep kieu sang float
        for j in range(n):
            row.append(float(A[i][j]))
        #Them ma tran don vi I
        for j in range(n):
            if i == j:
                row.append(1.0)
            else:
                row.append(0.0)
        matrix.append(row)
    # Thuc hien cac buoc Gauss-Jordan
    for i in range(n):
        # Tim pivoting (tim dong co gia tri lon nhat)
        max_row = i
        for k in range(i+1,n): # Duyet he cac dong ben duoi
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        # Doi dong
        temp = matrix[i]
        matrix[i] = matrix[max_row]
        matrix[max_row] = temp
        # Kiem tra tinh kha nghich xem ma tran co suy bien khong
        if abs(matrix[i][i]) < 1e-10:
            return None # Ma tran khong kha nghich
        # Chuan hoa pivot ve 1
        pivot = matrix[i][i]
        for j in range(2*n):
            matrix[i][j] /= pivot
        # Khu cac dong khac
        for k in range(n):
            if k != i:
                factor = matrix[k][i]
                for j in range(2*n):
```

```
                    matrix[k][j] -= factor * matrix[i][j]
    inv_A = []
    for i in range(n):
        row = []
        for j in range(n, 2 * n):
            row.append(matrix[i][j])
        inv_A.append(row)
    return inv_A
```

- **Độ phức tạp thời gian:** $O(n^3)$ do cấu trúc 3 vòng lặp lồng nhau.
- **Độ phức tạp không gian:** $O(n^2)$ để lưu trữ dữ liệu ma trận mở rộng.
- **Tính ổn định:** Việc sử dụng kỹ thuật chọn phần tử trội (Partial Pivoting) giúp giảm thiểu sai số làm tròn khi làm việc với số thực trên máy tính.

C.5.2. Hàm kiểm thử nghịch đảo

```
def verify_inverse_solution(A, x):
    A_np = np.array(A, dtype=float)
    x_np = np.array(x, dtype=float)

    try:
        A_numpy = np.linalg.inv(A_np)

        is_match = np.allclose(x_np, A_numpy)

        I_test = np.allclose(np.dot(A_np, x_np), np.eye(A_np.shape
    [0]))

        if is_match and I_test:
            print("Ket qua cua ban la dung.")
            return True
        else:
            print("Ket qua cua ban la sai.")
            print(f"Ket qua NumPy: {A_numpy}")
            return False

    except np.linalg.LinAlgError as e:
        print(f"Loi NumPy: {e}")
```

C.6 Kết quả thực nghiệm (Test Cases) và Đánh giá

Để đánh giá tính đúng đắn và độ ổn định của thuật toán Gauss-Jordan, ta tiến hành kiểm thử trên các trường hợp sau:

C.6.1. Test case 1: Ma trận đơn vị.

Ma trận đầu vào: $A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Kết quả: $A^{-1} = \begin{pmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{pmatrix}$

Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

Nhận xét: Thuật toán trả về chính ma trận đơn vị, phù hợp với lý thuyết.

C.6.2. Test case 2: Ma trận suy biến.

Ma trận đầu vào: $A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$

Kết quả: Ma trận không khả nghịch.

Kết quả kiểm thử:

```
Loi NumPy: Singular matrix
```

Nhận xét: Thuật toán phát hiện pivot bằng 0 và dừng đúng (TH1).

C.6.3. Test case 3: Ma trận có pivot nhỏ.

Ma trận đầu vào: $A = \begin{pmatrix} 10^{-8} & 1 & 2 \\ 1 & 10^{-8} & 3 \\ 2 & 3 & 10^{-8} \end{pmatrix}$

Kết quả: $A^{-1} \approx \begin{pmatrix} -0.75 & 0.50 & 0.25 \\ 0.50 & -0.33 & 0.17 \\ 0.25 & 0.17 & -0.08 \end{pmatrix}$

Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

Nhận xét: Nhờ kỹ thuật *Partial Pivoting*, thuật toán vẫn cho kết quả chính xác dù pivot rất nhỏ.

C.6.4. Test case 4: Ma trận Hilbert.

Ma trận đầu vào: $A = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{pmatrix}$

Kết quả: $A^{-1} \approx \begin{pmatrix} 9.00 & -36.00 & 30.00 \\ -36.00 & 192.00 & -180.00 \\ 30.00 & -180.00 & 180.00 \end{pmatrix}$

Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

Nhận xét: Thuật toán xử lý tốt với ma trận có điều kiện kém (ill-conditioned).

C.6.5. Test case 5: Ma trận hoán vị.

Ma trận đầu vào: $A = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \\ 2 & 3 & 1 \end{pmatrix}$

Kết quả: $A^{-1} \approx \begin{pmatrix} -0.28 & 0.39 & 0.06 \\ 0.06 & -0.28 & 0.39 \\ 0.39 & 0.06 & -0.28 \end{pmatrix}$

Kết quả kiểm thử:

```
Ket qua cua ban la dung.
```

Nhận xét: Thuật toán cho kết quả chính xác với ma trận hoán vị tuần hoàn.

C.6.6. Đánh giá

- Thuật toán hoạt động chính xác với các ma trận khả nghịch.
- Phát hiện đúng ma trận suy biến.
- Kỹ thuật Partial Pivoting giúp tăng độ ổn định số.
- Xử lý tốt các trường hợp đặc biệt như ma trận Hilbert.

D Tính hạng và cơ sở của các không gian

D.1 Mục tiêu và Bài toán

Nhiệm vụ của phần này là xây dựng các thuật toán để đưa một ma trận bất kỳ về dạng bậc thang (REF) và bậc thang rút gọn (RREF). Dựa trên kết quả RREF, chương trình sẽ xác định hạng (rank) của ma trận A, đồng thời tìm ra các hệ cơ sở đại diện cho ba không gian quan trọng: không gian dòng (row space), không gian cột (column space) và không gian nghiệm (null space).

D.2 Cơ sở lý thuyết

Định nghĩa D.2.1 (Dạng bậc thang dòng - REF). Ma trận **U** ở *dạng bậc thang dòng* (Row Echelon Form, viết tắt REF) nếu:

1. Mọi dòng toàn số 0 nằm phía dưới các dòng khác 0.
2. Phần tử chỉ huy (pivot) của mỗi dòng khác 0 nằm bên phải pivot của dòng ngay trên nó.

Định nghĩa D.2.2 (Dạng bậc thang dòng rút gọn — RREF). Ma trận $\mathbf{R}$ ở *dạng bậc thang dòng rút gọn* (Reduced REF, viết tắt RREF) nếu ngoài điều kiện REF còn có:

3. Mỗi pivot bằng 1.
4. Mỗi cột chứa pivot có tất cả các phần tử còn lại bằng 0.

Định nghĩa D.2.3 (Hạng ma trận). *Hạng* (rank) của ma trận $\mathbf{A}$, ký hiệu $\operatorname{rank}(\mathbf{A})$, bằng số dòng khác 0 trong dạng REF của $\mathbf{A}$, hoặc tương đương, bằng số cột pivot.

Từ dạng RREF, ta xác định được ba không gian con quan trọng:

- **Không gian cột** (column space) $\mathscr{C}(\mathbf{A})$: sinh bởi các cột pivot của $\mathbf{A}$ gốc.
- **Không gian dòng** (row space) $\mathscr{R}(\mathbf{A})$: sinh bởi các dòng khác 0 trong RREF.
- **Không gian nghiệm** (null space) $\mathscr{N}(\mathbf{A})$: tập nghiệm của $\mathbf{Ax} = \mathbf{0}$.

D.3 Thuật toán

D.3.1. Biến đổi về dạng bậc thang

- **Khởi tạo:** Tạo bản sao B của ma trận A và đặt các chỉ số dòng chốt $pr = 0$, cột chốt $pc = 0$.
- **Quá trình khử Gauss:** Khi $pr < m$ và $pc < n$:
  1. **Tìm phần tử trội (Partial Pivoting):** Tìm dòng p (từ dòng pr đến $m-1$) sao cho giá trị tuyệt đối của phần tử tại cột pc là lớn nhất:
  $$|B[p][pc]| = \max_{pr \le i < m} |B[i][pc]|$$
  2. **Hoán đổi dòng:** Nếu tìm thấy $p \neq pr$, hoán đổi dòng p và dòng pr để đưa phần tử trội lên vị trí chốt.

3. **Kiểm tra chốt:** Nếu $|B[pr][pc]| < \varepsilon$, bỏ qua cột này và chuyển sang $pc+1$ (do các phần tử bên dưới đều bằng 0).
4. **Biến đổi dòng:** Với mỗi dòng i nằm dưới dòng chốt ($i > pr$), tính hệ số khử:

$$\text{factor} = \frac{B[i][pc]}{B[pr][pc]}$$

   Cập nhật dòng i theo công thức: $R_i = R_i - \text{factor} \times R_{pr}$, cụ thể cho từng phần tử j:

$$B[i][j] = B[i][j] - \text{factor} \times B[pr][j]$$

5. **Cập nhật:** Tăng pr và pc để xét vị trí chốt tiếp theo.

D.3.2. Biến đổi về dạng bậc thang rút gọn

- **Giai đoạn 1: Biến đổi về dạng REF:** Sử dụng hàm `to_ref(A)` để đưa ma trận về dạng bậc thang cơ bản.
- **Giai đoạn 2: Khử ngược (Back-substitution):** Duyệt từ dòng dưới cùng lên trên để xử lý từng dòng chốt:
  1. **Xác định chốt:** Tìm vị trí phần tử chốt đầu tiên (pr, pc) khác 0 (lớn hơn ε) của dòng hiện tại tính từ trái sang.
  2. **Chuẩn hóa chốt (Leading 1):** Chia toàn bộ dòng chốt cho giá trị của phần tử chốt đó để biến phần tử chốt thành số 1:

$$B[pr][j] = \frac{B[pr][j]}{B[pr][pc]} \quad (\text{với } j \geq pc)$$

  3. **Triệt tiêu phía trên:** Với mỗi dòng i nằm phía trên dòng chốt ($i < pr$), thực hiện phép biến đổi dòng để đưa phần tử tại cột pc về bằng 0:

$$B[i][j] = B[i][j] - B[i][pc] \times B[pr][j]$$

- **Kết quả:** Sau khi duyệt hết các dòng, ma trận thu được sẽ thỏa mãn các điều kiện của dạng bậc thang rút gọn.

D.3.3. Tìm hạng và các cơ sở của ma trận

- **Biến đổi ma trận:** Chuyển ma trận A về dạng bậc thang rút gọn $\text{rref}(A)$ để xác định các vị trí chốt (pivots).
- **Xác định hạng (r):** Hạng của ma trận được tính bằng số lượng dòng khác không trong $\text{rref}(A)$.
- **Cơ sở dòng:** Tập hợp r dòng đầu tiên (các dòng khác không) của ma trận $\text{rref}(A)$.
- **Cơ sở cột:** Tập hợp các cột của ma trận A gốc tại các vị trí tương ứng với các cột chứa phần tử chốt (leading 1) trong $\text{rref}(A)$.
- **Cơ sở không gian nghiệm (Null space):** Giải hệ phương trình thuần nhất $Ax = 0$ bằng cách sử dụng $\text{rref}(A)$:
  1. Phân loại các biến thành biến phụ thuộc (ứng với cột có chốt) và biến tự do.
  2. Biểu diễn các biến phụ thuộc theo các biến tự do:

$$x_{pivot} = -\sum_{j \in \text{free}} \text{rrefA}_{i,j} \cdot x_j$$

  3. Với mỗi biến tự do, gán giá trị bằng 1 và các biến tự do còn lại bằng 0 để tìm một vector cơ sở của không gian nghiệm.

D.4 Mã giả

Algorithm 5 Chuyển ma trận A về dạng REF (`to_ref`)

```
Input: Ma trận A kích thước m × n
Output: Ma trận B ở dạng REF
B ← bản sao độc lập của A
pr ← 0, pc ← 0 Lưu vị trí pivot hiện tại
while pr < m and pc < n do
    p ← pr
    for i = pr + 1 to m − 1 do                ▷ Tìm dòng có phần tử chốt lớn nhất
        if |B[i][pc]| > |B[p][pc]| then
            p ← i
        end if
    end for
    if p ≠ pr then
        Hoán đổi dòng pr và dòng p của B
    end if
    if |B[pr][pc]| < ε then                   ▷ Không có phần tử chốt khác 0
        pc ← pc + 1
        continue
    end if
    for i = pr + 1 to m − 1 do                ▷ Khử các phần tử nằm dưới chốt
        factor ← B[i][pc]/B[pr][pc]
        for j = pc to n − 1 do
            B[i][j] ← B[i][j] − factor × B[pr][j]
        end for
    end for
    pr ← pr + 1, pc ← pc + 1
end while
return B
```

Algorithm 6 Chuyển ma trận A về dạng RREF (`to_rref`)

```
Input: Ma trận A
Output: Ma trận B ở dạng RREF
B ← to_ref(A)
pr ← m − 1, pc ← n − 1
while pr ≥ 0 do
    found ← False
    Tìm vị trí chốt (i, j) từ dưới lên trên, phải sang trái sao cho |B[i][j]| > ε
    if Tìm thấy chốt then
        Cập nhật pr ← i, pc ← j và found ← True
    end if
    if found then
        for j = n − 1 down to pc do                    ▷ Tạo leading 1
            B[pr][j] ← B[pr][j]/B[pr][pc]
        end for
        for i = pr − 1 down to 0 do                    ▷ Khử các dòng nằm trên chốt
            if |B[i][pc]| > ε then
                factor ← B[i][pc]
                for j = pc to n − 1 do
                    B[i][j] ← B[i][j] − factor × B[pr][j]
                end for
            end if
        end for
    end if
    pr ← pr − 1, pc ← pc − 1
end while
return B
```

Algorithm 7 Tìm hạng và các cơ sở (`rank_and_basis`)

Input: Ma trận A
Output: Hạng r, tập cơ sở cột, tập cơ sở dòng, tập cơ sở không gian nghiệm
rrefA $\leftarrow$ `to_rref`(A) ▷ Tính dạng rref của A
Tính hạng:
$r \leftarrow 0$
for $i = m-1$ **down to** 0 **do**
  if Dòng i có phần tử khác 0 **then**
    $r \leftarrow i+1$
    break
  end if
end for
Cơ sở dòng: Thêm r dòng đầu tiên của rrefA vào danh sách.
Cơ sở cột: Lấy các cột trong ma trận A ứng với các cột chứa "leading 1" trong rrefA
Cơ sở nghiệm (Null space):
Khởi tạo ma trận C kích thước $n \times n$ với đường chéo chính bằng 1.
Ghi nhận danh sách `pivots` chứa vị trí các cột có chốt (biến phụ thuộc).
for Mỗi dòng i có chốt tại cột `pivot` trong rrefA **do**
  for $j = \text{pivot}+1$ **to** $n-1$ **do**
    $C[j][\text{pivot}] \leftarrow -\text{rrefA}[i][j]$ ▷ Biểu diễn biến phụ thuộc theo biến tự do
  end for
end for
Cơ sở nghiệm bao gồm các dòng j của ma trận C mà $j \notin$ `pivots`.
return r, cơ sở cột, cơ sở dòng, cơ sở nghiệm

D.5 Cài đặt và Các hàm chính

D.5.1. Hàm chuyển ma trận về dạng REF

```python
def to_ref(A: list[list]) -> list[list]:
    B = copy.deepcopy(A)
    epsilon = 1e-12
    m = len(B)
    n = len(B[0])
    pr =pc = 0

    while pr < m and pc < n:
        # Hoan voi dong co |A_ik|max
        p = pr
        for i in range(pr+1, m):
            if abs(B[i][pc]) > abs(B[p][pc]):
                p=i

        if p != pr:
```

```
            B[pr], B[p] = B[p], B[pr]

        # Khong co pivot
        if abs(B[pr][pc]) < epsilon:
            pc += 1
            continue

        # Khu cac dong duoi
        for i in range(pr+1, m):
            factor = B[i][pc]/B[pr][pc]
            for j in range(pc,n):
                B[i][j] -= factor*B[pr][j]

        pr+=1
        pc+=1
    return B
```

D.5.2. Hàm chuyển ma trận về dạng RREF

```
    def to_rref(A: list[list]) -> list[list]:
    B = to_ref(A)

    epsilon = 1e-12
    m = len(B)
    n = len(B[0])
    pr = m-1
    pc = n-1

    while(pr >=0):
        # Tim pivot
        found = False
        for i in range(pr,-1, -1):
            if found: break

            for j in range(pc, -1, -1):
                if abs(B[i][j]) > epsilon:
                    pr, pc = i, j
                    found = True

        if found:
            # Leading 1
            for j in range(n-1, pc-1, -1):
                B[pr][j]/=B[pr][pc]

```

```
            # Khu cac dong tren
            for i in range(pr-1, -1, -1):
                if abs(B[i][pc]) > epsilon:
                    factor = B[i][pc]
                    for j in range(pc, n):
                        B[i][j] -= factor*B[pr][j]

        pr -= 1
        pc -= 1
    return B
```

D.5.3. Hàm tìm hạng và các cơ sở

```
def rank_and_basis(A: list[list]) -> tuple[int, list[list[float]],
    list[list[float]], list[list[float]]]:
    rrefA = to_rref(A)

    m = len(A)
    n = len(A[0])
    epsilon = 1e-12

    row_basis = []
    col_basis = []
    null_basis = []
    # Tinh rank
    rank = 0
    for i in range(m-1, -1, -1):
        if rank > epsilon: break

        for j in range(0,n):
            if abs(rrefA[i][j]) > epsilon:
                rank=i+1
                break
    # Co so dong
    row_basis = [rrefA[i] for i in range(rank)]
    # Co so cot
    pc = 0
    for i in range(rank):
        # Tim pivot
        while pc < n and abs(rrefA[i][pc] - 1) > epsilon:
            pc+=1

        if pc >= n or abs(rrefA[i][pc] - 1) > epsilon: break
        # Them cot
```

```
        col = []
        for i_ in range(m):
            col.append(A[i_][pc])
        col_basis.append(col)

    # Co so nghiem
    C = [[0] * n for _ in range(n)]
    pivots = [] # Danh giau pivot (cac an phu thuoc)
    for j in range(n): C[j][j] = 1
    for i in range(m):
        pivot = -1
        for j in range(n):
            if abs(rrefA[i][j]) > epsilon:
                pivot = j
                pivots.append(j)
                break
        if pivot==-1: continue
        for j in range(pivot+1, n):
            C[j][pivot] = -rrefA[i][j]

    # Giu lai bien tu do
    null_basis = [C[j] for j in range(n) if j not in pivots]

    return rank, col_basis, row_basis, null_basis
```

- **Độ phức tạp thời gian:** Khối lượng tính toán chủ yếu nằm ở lời gọi hàm `to_rref(A)`, do đó độ phức tạp là $O(n^3)$. Các thao tác quét mảng sau đó để trích xuất hạng và cơ sở chỉ tốn thêm chi phí $O(n^2)$.
- **Độ phức tạp không gian:** $O(n \cdot m + n^2)$ để lưu trữ ma trận `rrefA` và khởi tạo ma trận phụ C kích thước $n \times n$ nhằm tính toán cơ sở cho không gian nghiệm (Null space).
- **Tính chính xác:** Thay vì so sánh trực tiếp với 0, thuật toán sử dụng hằng số $\varepsilon = 10^{-12}$ để triệt tiêu các sai số dấu phẩy động, đảm bảo việc xác định thứ hạng (rank) và nhận diện các biến tự do (free variables) luôn chính xác.

D.5.4. Hàm kiểm thử hạng

```
def verify_rank_solution(A, x):
    A_np = np.array(A, dtype=float)

```

```
    try:
        x_np = np.linalg.matrix_rank(A_np)

        if x == x_np:
            print("Ket qua cua ban la dung.")
            return True
        else:
            print("Ket qua cua ban la sai.")
            print(f"Ket quaNumPy: {x_np}")
            return False

    except np.linalg.LinAlgError as e:
        print(f"Loi NumPy: {e}")
```

D.5.5. Hàm kiểm thử cơ sở của không gian cột

```
def verify_column_space(A, x):
    A_np = np.array(A, dtype=float)

    try:
        rank_A = np.linalg.matrix_rank(A_np)

        if len(x) != rank_A:
            print("Sai so chieu khong gian cot.")
            print(f"Ket qua NumPy: {rank_A}")
            return False

        x_np = np.array(x, dtype=float).T
        A_combined = np.column_stack((A_np, x_np))
        A_combined_rank = np.linalg.matrix_rank(A_combined)

        if A_combined_rank == rank_A:
            print("Khong gian cot chinh xac.")
            return True
        else:
            print("Tap vector khong sinh ra khong gian cot cua A."
    )
            return False
    except np.linalg.LinAlgError as e:
        print(f"Loi NumPy: {e}")
```

D.5.6. Hàm kiểm thử cơ sở của không gian dòng

```
def verify_row_space(A, x):
    A_np = np.array(A, dtype=float)
```

```python

    try:
        rank_A = np.linalg.matrix_rank(A_np)

        if len(x) != rank_A:
            print("Sai so chieu khong gian dong.")
            print(f"Ket qua NumPy: {rank_A}")
            return False

        A_T = A_np.T
        x_np = np.array(x, dtype=float).T
        A_combined = np.column_stack((A_T, x_np))
        A_combined_rank = np.linalg.matrix_rank(A_combined)

        if A_combined_rank == rank_A:
            print("Khong gian dong chinh xac.")
            return True
        else:
            print("Tap vector khong sinh ra khong gian dong cua A.")
            return False
    except np.linalg.LinAlgError as e:
        print(f"Loi NumPy: {e}")
```

D.5.7. Hàm kiểm thử cơ sở của không gian nghiệm

```python
def verify_null_space(A, x):
    A_np = np.array(A, dtype=float)
    A_cols = A_np.shape[1]
    rank_A = np.linalg.matrix_rank(A_np)

    dim = A_cols - rank_A
    if len(x) != dim:
        print("Sai so chieu.")
        print(f"So chieu NumPy: {dim}")
        return False

    for v in x:
        v_np = np.array(v, dtype=float)
        if not np.allclose(np.dot(A_np, v_np), 0):
            print("Vector trong co so khong thoa man Ax = 0.")
            return False

    print("Khong gian nghiem chinh xac.")
```

```
    return True
```

D.6 Kết quả thực nghiệm (Test Cases) và Đánh giá

D.6.1. Test Case 1: Ma trận vuông 3×3.

$$A_1 = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix}$$

- Hạng của ma trận: $r = 3$
- Cơ sở dòng: $\{[1,0,0],[0,1,0],[0,0,1]\}$
- Cơ sở cột: $\left\{ \begin{pmatrix} 1 \\ 0 \\ 5 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \\ 6 \end{pmatrix}, \begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix} \right\}$
- Cơ sở nghiệm: $\emptyset$ (Không gian nghiệm chỉ chứa vector $\mathbf{0}$)
- Kết quả kiểm thử:

```
    Ket qua cua ban la dung.
    Khong gian dong chinh xac.
    Khong gian cot chinh xac.
    Ket qua nghiem chinh xac.
```

D.6.2. Test Case 2: Ma trận vuông 3×3 (Phụ thuộc tuyến tính).

$$A_2 = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

- Hạng của ma trận: $r = 2$
- Cơ sở dòng: $\{[1,0,-1],[0,1,2]\}$
- Cơ sở cột: $\left\{ \begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}, \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix} \right\}$

- Cơ sở nghiệm: $\{[1,-2,1]\}$
- Kết quả kiểm thử:

```
    Ket qua cua ban la dung.
    Khong gian dong chinh xac.
    Khong gian cot chinh xac.
    Ket qua nghiem chinh xac.
```

D.6.3. Test Case 3: Ma trận chữ nhật 2×4.

$$A_3 = \begin{pmatrix} 1 & -2 & 0 & 4 \\ 3 & 1 & 1 & 0 \end{pmatrix}$$

- Hạng của ma trận: $r = 2$
- Cơ sở dòng: $\{[1,0,0.2857,0.5714],[0,1,0.1429,-1.7143]\}$
- Cơ sở cột: $\left\{ \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \begin{pmatrix} -2 \\ 1 \end{pmatrix} \right\}$
- Cơ sở nghiệm: $\{[-0.2857,-0.1429,1,0],[-0.5714,1.7143,0,1]\}$
- Kết quả kiểm thử:

```
    Ket qua cua ban la dung.
    Khong gian dong chinh xac.
    Khong gian cot chinh xac.
    Ket qua nghiem chinh xac.
```

D.6.4. Test Case 4: Ma trận chữ nhật 3×2.

$$A_4 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

- Hạng của ma trận: $r = 2$
- Cơ sở dòng: $\{[1,0],[0,1]\}$
- Cơ sở cột: $\left\{ \begin{pmatrix} 1 \\ 3 \\ 5 \end{pmatrix}, \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix} \right\}$

- Cơ sở nghiệm: $\emptyset$
- Kết quả kiểm thử:

```
    Ket qua cua ban la dung.
    Khong gian dong chinh xac.
    Khong gian cot chinh xac.
    Ket qua nghiem chinh xac.
```

D.6.5. Test Case 5: Ma trận 3×4 (Có dòng không và biến tự do).

$$A_5 = \begin{pmatrix} 1 & 1 & 2 & 2 \\ 2 & 2 & 4 & 4 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

- Hạng của ma trận: $r = 2$
- Cơ sở dòng: $\{[1,0,1,0],[0,1,1,2]\}$
- Cơ sở cột: $\left\{ \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix} \right\}$
- Cơ sở nghiệm: $\{[-1,-1,1,0],[0,-2,0,1]\}$
- Kết quả kiểm thử:

```
    Ket qua cua ban la dung.
    Khong gian dong chinh xac.
    Khong gian cot chinh xac.
    Ket qua nghiem chinh xac.
```

D.6.6. Đánh giá

Kết quả thực nghiệm khẳng định thuật toán đưa ma trận về dạng bậc thang rút gọn (RREF) hoạt động với độ tin cậy và chính xác toán học cao. Quá trình trích xuất hạng (rank) và cơ sở của các không gian dòng, cột, và không gian nghiệm được thực thi chặt chẽ, xử lý trơn tru các hệ tọa độ từ phụ thuộc tuyến tính đến các ma trận chứa dòng/cột toàn số không. Thuật toán thể hiện tính tổng quát tốt khi tính toán thành công cơ sở nghiệm (Null space) thông qua việc biểu diễn tham số linh hoạt, phản ánh đúng cấu trúc đại số và tính chất hình học của hệ thống.

5 Phần 2: Phân rã ma trận và trực quan hóa với Manim

A Phân rã LU (LU Decomposition)

A.1 Cơ sở lý thuyết và Mục tiêu

A.1.1. Đặt vấn đề

Cho một ma trận $A \in \mathbb{R}^{m\times n}$. Phép phân rã LU tiêu chuẩn nhằm tìm ma trận tam giác dưới L và ma trận tam giác trên U sao cho $A = LU$. Tuy nhiên, quá trình tính toán (khử Gauss) sẽ thất bại nếu phần tử chốt (pivot) bằng 0, hoặc gây ra sai số làm tròn nghiêm trọng nếu phần tử chốt quá nhỏ.

Để khắc phục vấn đề này, ta áp dụng kỹ thuật **chọn phần tử trội từng phần (Partial Pivoting)**. Thuật toán sẽ thực hiện hoán vị các hàng của ma trận ở mỗi bước khử sao cho phần tử chốt luôn mang giá trị tuyệt đối lớn nhất. Khi đó, phương trình phân rã được biểu diễn dưới dạng:

$$PA = LU$$

A.1.2. Cấu trúc các ma trận thành phần

Sau khi thực hiện phân rã, ta thu được ba ma trận với các tính chất sau:

- **Ma trận hoán vị $P \in \mathbb{R}^{m\times m}$:** Là ma trận vuông được khởi tạo từ ma trận đơn vị I, trong đó các hàng bị tráo đổi vị trí tương ứng với các phép hoán đổi hàng đã thực hiện trên A. Ma trận P có tính chất trực giao, tức là $P^{-1} = P^T$.
- **Ma trận tam giác dưới $L \in \mathbb{R}^{m\times m}$:** Có các phần tử trên đường chéo chính bằng 1 ($l_{i,i} = 1$). Các phần tử dưới đường chéo chính ($l_{i,j}$ với $i > j$) lưu trữ các hệ số nhân (multipliers) sinh ra trong quá trình khử.
- **Ma trận bậc thang $U \in \mathbb{R}^{m\times n}$:** Chứa kết quả của ma trận A sau khi thực hiện khử Gauss tiến. Các phần tử thỏa mãn $u_{i,j} = 0$ với mọi $i > j$.

A.2 Thuật toán

Quá trình phân rã được thực hiện qua tối đa $\min(m,n)$ bước. Giả sử ta đang ở bước thứ k:

Bước A.2.1: Tìm phần tử trội (Pivoting).

Tìm chỉ số hàng p (với $k \leq p \leq m$) sao cho phần tử ở cột k đạt giá trị tuyệt đối lớn nhất:

$$|u_{p,k}| = \max_{k \leq i \leq m} |u_{i,k}|$$

Nếu $u_{p,k} = 0$, ta bỏ qua cột này và chuyển sang bước $k+1$.

Bước A.2.2: Hoán vị hàng.

Nếu $p \neq k$, ta thực hiện hoán đổi hàng k và hàng p:

- Hoán đổi hàng k và hàng p của U.
- Hoán đổi hàng k và hàng p của P.
- Hoán đổi các phần tử từ cột 1 đến cột $k-1$ của hàng k và hàng p trong ma trận L. Điều này là bắt buộc để các hệ số nhân tương ứng dịch chuyển cùng với phương trình gốc của chúng, đồng thời không làm hỏng đường chéo chính của L.

Bước A.2.3: Khử Gauss.

Với mỗi hàng i từ $k+1$ đến m:

1. Tính hệ số nhân: $l_{i,k} = \frac{u_{i,k}}{u_{k,k}}$.
2. Cập nhật phần tử của U: $u_{i,j} = u_{i,j} - l_{i,k} \cdot u_{k,j} \quad (\forall j \geq k)$.

A.3 Mã giả

Algorithm 8 Phân rã $PA = LU$ (Partial Pivoting)

Require: Ma trận $A \in \mathbb{R}^{m \times n}$
Ensure: Ma trận hoán vị P, ma trận tam giác dưới L, ma trận tam giác trên U

Khởi tạo $U \leftarrow A$
Khởi tạo $L \leftarrow I_m$ ▷ Ma trận đơn vị cấp m
Khởi tạo $P \leftarrow I_m$
for $k = 0$ **to** $\min(m,n) - 1$ **do**
  // Tìm phần tử trội
  Tìm chỉ số $p \geq k$ sao cho $|U[p,k]| = \max_{i=k}^{m-1} |U[i,k]|$
  if $U[p,k] == 0$ **then**
    continue ▷ Bỏ qua nếu cột toàn số 0
  end if
  // Hoán vị hàng
  if $p \neq k$ **then**
    Hoán đổi $U[k,*]$ và $U[p,*]$
    Hoán đổi $P[k,*]$ và $P[p,*]$
    if $k > 0$ **then**
      Hoán đổi $L[k,0:(k-1)]$ và $L[p,0:(k-1)]$
    end if
  end if
  // Khử Gauss
  for $i = k+1$ **to** $m-1$ **do**
    $L[i,k] \leftarrow U[i,k]/U[k,k]$
    for $j = k$ **to** $n-1$ **do**
      $U[i,j] \leftarrow U[i,j] - L[i,k] \times U[k,j]$
    end for
  end for
end for
return P, L, U

A.4 Cài đặt và Các hàm chính

```python
class LUDecomposition:
    @staticmethod
    def create_identity_matrix(size):
        return [[1.0 if i == j else 0.0 for j in range(size)] for i in range(size)]

    @staticmethod
    def copy_matrix(matrix):
        return [row[:] for row in matrix]

```

```
    def __init__(self,A):
        if not A or not A[0]:
            raise ValueError("Ma tran dau vao khong duoc rong")
        self.m=len(A)
        self.n=len(A[0])
        self.A = self.copy_matrix(A)
        self.P = None
        self.L = None
        self.U = None

    def decompose(self):
        m, n = self.m, self.n

        # P,L la ma tran vuong mxm
        P = self.create_identity_matrix(m)
        L = self.create_identity_matrix(m)

        # U la ma tran mxn
        U=self.copy_matrix(self.A)

        # So buoc khu toi da
        limit=min(m,n)

        for i in range(limit):
            pivot_row = i
           #  Tim phan tu lon nhat trong cot i, tu hang i den hang m - 1
            max_val = abs(U[i][i])
            for k in range(i + 1, m):
                if abs(U[k][i]) > max_val:
                    max_val = abs(U[k][i])
                    pivot_row = k

            # Neu toan bo cot duoi duong cheo deu la 0, bo qua khong can khu
            if max_val == 0:
                continue

            # Doi hang neu can
            if pivot_row != i:
                # Doi hang trong U
                U[i], U[pivot_row] = U[pivot_row], U[i]
                # Doi hang trong P
                P[i], P[pivot_row] = P[pivot_row], P[i]
```

```python

                # Doi hang trong L (chi doi cac cot ben trai phan tu duong cheo i)
                for k in range(i):
                    L[i][k], L[pivot_row][k] = L[pivot_row][k], L[i][k]

            # 2. Khu Gauss
            for j in range(i + 1, m):
                factor = U[j][i] / U[i][i]
                L[j][i] = factor

                # Cap nhat cac phan tu con lai tren hang j cua U
                for k in range(i, n):
                    U[j][k] -= factor * U[i][k]

        self.P, self.L, self.U = P, L, U
        return P, L, U
```

A.5 Kết quả thực nghiệm (Test Cases) và Đánh giá

Thuật toán phân rã $PA = LU$ đã được cài đặt thực nghiệm bằng ngôn ngữ Python và được đối chiếu độ chính xác với hàm `scipy.linalg.lu` của thư viện `SciPy` chuyên dụng. Các kịch bản kiểm thử (Test Cases) được thiết kế bao quát các dạng ma trận khác nhau để kiểm chứng tính đúng đắn và độ ổn định của thuật toán. Tất cả các trường hợp đều thỏa mãn hằng đẳng thức $P \cdot A = L \cdot U$.

A.5.1. Ma trận vuông tiêu chuẩn (Baseline Case)

Trong kịch bản kiểm thử ma trận vuông 3×3, thuật toán tự cài đặt cho kết quả hoàn hảo:

- **Độ chính xác:** Đẳng thức $P \cdot A = L \cdot U$ được thỏa mãn.
- **Tính đồng nhất:** Cả ba ma trận P, L, U thu được đều trùng khớp hoàn toàn (100%) với kết quả từ thư viện `SciPy`. Điều này khẳng định logic chọn phần tử trội và các bước khử Gauss đã được cài đặt đúng theo tiêu chuẩn.
- **Test Case 1: Ma trận vuông** (3×3)

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix} \quad \Rightarrow \quad P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ -0.5 & 1 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 4 & -6 & 0 \\ 0 & 4 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

A.5.2. Ma trận chữ nhật Đây là điểm khác biệt cốt lõi giữa thuật toán tự cài đặt và thư viện như `SciPy`:

- **Trường hợp $A \in \mathbb{R}^{4\times 3}$ (Nhiều hàng hơn cột):** Thuật toán trả về phân rã dạng đầy đủ (Full Form) với $L_{4\times 4}$ và $U_{4\times 3}$. Trong khi đó, `SciPy` trả về dạng rút gọn (Economy Form) với $L_{4\times 3}$ và $U_{3\times 3}$ nhằm tối ưu hóa bộ nhớ do hàng cuối cùng của U bị khử hoàn toàn về 0. Cả hai cách tiếp cận đều đúng về mặt toán học.

  Test Case 2: Ma trận chữ nhật (4×3)

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & -4 & 6 \\ 3 & -9 & -3 \\ -2 & 4 & -1 \end{bmatrix} \quad \Rightarrow \quad P = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.3333 & 1 & 0 & 0 \\ 0.6667 & 0.4 & 1 & 0 \\ -0.6667 & -0.4 & -0.2187 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 3 & -9 & -3 \\ 0 & 5 & 4 \\ 0 & 0 & 6.4 \\ 0 & 0 & 0 \end{bmatrix}$$

- **Trường hợp $A \in \mathbb{R}^{2\times 4}$ (Nhiều cột hơn hàng):** Cả hai thuật toán đều đồng nhất về kết quả và kích thước ma trận thành phần $(P_{2\times 2}, L_{2\times 2}, U_{2\times 4})$.

  Test Case 3: Ma trận chữ nhật (2×4)

$$A = \begin{bmatrix} 2 & 4 & -1 & 5 \\ -4 & -5 & 3 & -8 \end{bmatrix} \quad \Rightarrow \quad P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 \\ -0.5 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} -4 & -5 & 3 & -8 \\ 0 & 1.5 & 0.5 & 1 \end{bmatrix}$$

A.5.3. Các trường hợp đặc biệt (Edge Cases)

Thuật toán chứng minh được sự bền bỉ khi xử lý mượt mà các ma trận có tính chất cực đoan:

- **Ma trận chứa cột toàn số** 0 **(Ma trận suy biến):** Thuật toán nhận diện được phần

tử trội cực đại bằng 0, tự động bỏ qua bước khử tại cột này mà không sinh ra lỗi chia cho 0 (ZeroDivisionError).

Test Case 4: Ma trận chứa cột suy biến

$$A = \begin{bmatrix} 0 & 2 & 3 \\ 0 & 4 & 6 \\ 0 & 6 & 9 \end{bmatrix} \quad \Rightarrow \quad P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0.6667 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & 2 & 3 \\ 0 & 6 & 9 \\ 0 & 0 & 0 \end{bmatrix}$$

- **Ma trận tam giác trên có sẵn:** Thuật toán đủ thông minh để nhận diện ma trận đã ở trạng thái tối ưu, do đó không thực hiện phép hoán vị hay khử dư thừa nào. Các ma trận P và L được giữ nguyên là ma trận đơn vị I.

  Test Case 5: Ma trận tam giác trên (Đã chuẩn sẵn)

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & -1 \\ 0 & 0 & 7 \end{bmatrix} \quad \Rightarrow \quad P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & -1 \\ 0 & 0 & 7 \end{bmatrix}$$

- **Ma trận đường chéo phụ:** Thuật toán thực hiện hoán vị liên tục và chính xác để đưa các phần tử khác 0 lên đường chéo chính, chứng minh tính đúng đắn của logic tráo đổi cột trong ma trận L.

  Test Case 6: Ma trận đường chéo phụ (Yêu cầu hoán vị liên tục)

$$A = \begin{bmatrix} 0 & 0 & 3 \\ 0 & 2 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad \Rightarrow \quad P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

A.5.4. Đánh giá

Kết quả thực nghiệm cho thấy thuật toán không chỉ đảm bảo độ chính xác toán học tuyệt đối mà còn có khả năng bao quát toàn bộ các kịch bản ma trận trong đại số tuyến tính thực hành.

B Phân rã QR (QR Decomposition)

B.1 Cơ sở lý thuyết và Mục tiêu

Phân tích ma trận A thành tích của ma trận trực giao Q và ma trận tam giác trên R ($A = QR$) bằng phương pháp trực giao hóa Gram-Schmidt.

Dạng tổng quát: $\mathbf{A} = \mathbf{QR}$

- $\mathbf{A} \in \mathbb{R}^{m\times n}$
- $\mathbf{Q} \in \mathbb{R}^{m\times n}$: Ma trận trực chuẩn ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$)
- $\mathbf{R} \in \mathbb{R}^{n\times n}$: Ma trận tam giác trên với các phần tử trên đường chéo chính đều dương, khả nghịch.

Phương pháp cài đặt: Gram–Schmidt cổ điển (CGS) hoặc Householder reflections.
Công thức Gram–Schmidt:

$$\mathbf{q}_k = \frac{\mathbf{a}_k - \sum_{j=1}^{k-1}(\mathbf{a}_k^T\mathbf{q}_j)\mathbf{q}_j}{\left\|\mathbf{a}_k - \sum_{j=1}^{k-1}(\mathbf{a}_k^T\mathbf{q}_j)\mathbf{q}_j\right\|}. \tag{7}$$

B.2 Thuật toán

- **Tìm Q:**
  1. Với mỗi cột i của A, chuyển thành dòng và lưu vào u_i.
  2. Trực giao hóa các u:
  $$v_i = u_i - \sum_{j=1}^{i-1} \frac{\langle u_i, v_j\rangle}{||v_j||^2} \cdot v_j$$
  3. Chuẩn hóa các v: $q_i = \frac{v_i}{||v_i||}$.
  4. Xây dựng Q từ q:
  $$Q = \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{pmatrix}^T$$

- **Tìm R:**

  1. Khởi tạo ma trận R kích thước $n \times n$ với các phần tử bằng 0.
  2. Tại mỗi vị trí i, j (với $j \geq i$): Tính $r_{ij} = \langle u_j, q_i \rangle$ *(với u_j là cột thứ j của A).*
  3. Xây dựng ma trận R hoàn chỉnh:

$$R = \begin{pmatrix} \langle u_1, q_1 \rangle & \langle u_2, q_1 \rangle & \cdots & \langle u_n, q_1 \rangle \\ 0 & \langle u_2, q_2 \rangle & \cdots & \langle u_n, q_2 \rangle \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \langle u_n, q_n \rangle \end{pmatrix}$$

B.3 Mã giả

Algorithm 9 Phân tích QR (`qr_decomposition`)

Input: Ma trận A kích thước $m \times n$
Output: Ma trận trực giao Q $(m \times n)$ và ma trận tam giác trên R $(n \times n)$
if $n \times m == 0$ **then**
    return [], []
end if
$Q \leftarrow \emptyset$
$u \leftarrow A^T$ ▷ Chuyển các cột của A thành các dòng để dễ xử lý
for $i = 0$ **to** $n - 1$ **do** ▷ Trực giao hóa Gram-Schmidt
    $v_i \leftarrow$ bản sao của $u[i]$
    for mỗi vector $_v \in Q$ **do**
        len_v $\leftarrow \langle _v, _v \rangle$
        if len_v $> \varepsilon$ **then**
            proj $\leftarrow \langle v_i, _v \rangle /$ len_v
            $v_i \leftarrow v_i -$ proj $\times _v$
        end if
    end for
    Thêm v_i vào danh sách Q
end for
for $i = 0$ **to** $n - 1$ **do** ▷ Chuẩn hóa các vector trong Q
    $v_i \leftarrow Q[i]$
    norm $\leftarrow \sqrt{\sum_{x \in v_i} x^2}$
    if norm $> \varepsilon$ **then**
        $Q[i] \leftarrow [x/\text{norm for } x \in v_i]$
    else
        $Q[i] \leftarrow$ vector $\mathbf{0}$ kích thước m
    end if
end for
Khởi tạo ma trận R kích thước $n \times n$ với các phần tử bằng 0.0
for $i = 0$ **to** $n - 1$ **do** ▷ Tính ma trận R
    for $j = i$ **to** $n - 1$ **do**
        $R[i][j] \leftarrow \langle u[j], Q[i] \rangle$
    end for
end for
return Q^T, R ▷ Trả về Q dưới dạng cột và R

B.4 Cài đặt và Các hàm chính

```python
def inner_product(v1: list, v2: list) -> float:
    if len(v1) != len(v2):
        raise ValueError("Hai vector phai co cung do dai")
```

```python

    return sum(x * y for x, y in zip(v1, v2))

def sum_vector(v1: list, v2: list) -> list:
    if len(v1) != len(v2):
        raise ValueError("Hai vector phai co cung do dai")

    return [x + y for x, y in zip(v1, v2)]

def transpose(A: list[list]) -> list[list]:
    n = len(A) # So hang cua ma tran goc
    m = len(A[0]) # So cot cua ma tran goc

    if n*m == 0:
        return []

    res = [[A[i][j] for i in range(n)] for j in range(m)] # Hoan doi chi so hang va cot
    return res

def qr_decomposition(A: list[list]) -> tuple[list[list], list[list]]:
    m = len(A) # So hang cua ma tran goc
    n = len(A[0]) # So cot cua ma tran goc

    if n*m == 0:
        return [], []

    Q = []
    u = transpose(A) # Chuyen cac cot thanh hang, n * m

    for i in range(n):
        v_i = u[i].copy()
        for _v in Q: # Truc quan hoa cac vector trong Q
            _v_len = inner_product(_v, _v)
            if _v_len > epsilon:
                # v_i -= <v_i, _v> / <_v, _v> * _v

                proj = inner_product(v_i, _v) / _v_len

                v_i = sum_vector(
                        v_i,
                        [-proj * _v[j] for j in range(m)]
                    )
```

```python

        Q.append(v_i)

    # Chuan hoa cac vector trong Q(nxm)
    for i in range(n):
        v_i = Q[i]
        norm_v_i = sum(x**2 for x in v_i) ** 0.5
        if norm_v_i > epsilon:
            Q[i] = [x / norm_v_i for x in v_i]
        else:
            Q[i] = [0.0] * m

    # Tinh R(nxn), moi phan tu R[i][j] = <u[j], Q[i]> neu i <= j
    R = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            R[i][j] = inner_product(u[j], Q[i])

    return transpose(Q), R
```

B.5 Kết quả thực nghiệm (Test Cases) và Đánh giá

Thuật toán phân rã $A = QR$ đã được cài đặt thực nghiệm bằng ngôn ngữ Python và được đối chiếu độ chính xác với hàm `numpy.linalg.qr` của thư viện `NumPy` chuyên dụng. Các kịch bản kiểm thử (Test Cases) được thiết kế bao quát các dạng ma trận khác nhau để kiểm chứng tính đúng đắn và độ ổn định của thuật toán. Tất cả các trường hợp đều thỏa mãn hằng đẳng thức $A = Q \cdot R$, đồng thời đảm bảo tính trực giao của ma trận Q ($Q^T \cdot Q = I$) và R là ma trận tam giác trên.

B.5.1. Test Case 1: Ma trận vuông 3×3.

Ma trận đầu vào A_1:

$$A_1 = \begin{pmatrix} 12 & -51 & 4 \\ 6 & 167 & -68 \\ -4 & 24 & -41 \end{pmatrix}$$

Ma trận trực giao Q:

$$Q = \begin{pmatrix} 0.8571 & -0.3943 & -0.3314 \\ 0.4286 & 0.9029 & 0.0343 \\ -0.2857 & 0.1714 & -0.9429 \end{pmatrix}$$

Ma trận tam giác trên R:

$$R = \begin{pmatrix} 14.0 & 21.0 & -14.0 \\ 0.0 & 175.0 & -70.0 \\ 0.0 & 0.0 & 35.0 \end{pmatrix}$$

Ma trận tái tạo ($Q \times R$):

$$QR = \begin{pmatrix} 12.0 & -51.0 & 4.0 \\ 6.0 & 167.0 & -68.0 \\ -4.0 & 24.0 & -41.0 \end{pmatrix}$$

B.5.2. Test Case 2: Ma trận chữ nhật 4×2.

Ma trận đầu vào A_2:

$$A_2 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Ma trận trực giao Q:

$$Q = \begin{pmatrix} 0.1091 & 0.8295 \\ 0.3273 & 0.4392 \\ 0.5455 & 0.0488 \\ 0.7638 & -0.3416 \end{pmatrix}$$

Ma trận tam giác trên R:

$$R = \begin{pmatrix} 9.1652 & 10.9109 \\ 0.0 & 0.9759 \end{pmatrix}$$

Ma trận tái tạo ($Q \times R$):

$$QR = \begin{pmatrix} 1.0 & 2.0 \\ 3.0 & 4.0 \\ 5.0 & 6.0 \\ 7.0 & 8.0 \end{pmatrix}$$

B.5.3. Test Case 3: Ma trận suy biến 3×3.

Ma trận đầu vào A_3:

$$A_3 = \begin{pmatrix} 1 & 1 & 5 \\ 2 & 2 & 6 \\ 3 & 3 & 7 \end{pmatrix}$$

Ma trận trực giao Q:

$$Q = \begin{pmatrix} 0.2673 & 0.0 & 0.8729 \\ 0.5345 & 0.0 & 0.2182 \\ 0.8018 & 0.0 & -0.4364 \end{pmatrix}$$

Ma trận tam giác trên R:

$$R = \begin{pmatrix} 3.7417 & 3.7417 & 10.1559 \\ 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 2.6186 \end{pmatrix}$$

Ma trận tái tạo ($Q \times R$):

$$QR = \begin{pmatrix} 1.0 & 1.0 & 5.0 \\ 2.0 & 2.0 & 6.0 \\ 3.0 & 3.0 & 7.0 \end{pmatrix}$$

B.5.4. Test Case 4: Ma trận đơn vị 3×3.

Ma trận đầu vào A_4:

$$A_4 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Ma trận trực giao Q:

$$Q = \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{pmatrix}$$

Ma trận tam giác trên R:

$$R = \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{pmatrix}$$

Ma trận tái tạo ($Q \times R$):

$$QR = \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{pmatrix}$$

B.5.5. Test Case 5: Ma trận vuông 4×4.

Ma trận đầu vào A_5:

$$A_5 = \begin{pmatrix} 4 & 1 & 3 & 2 \\ 1 & 5 & 0 & 1 \\ 3 & 0 & 6 & 2 \\ 2 & 1 & 2 & 7 \end{pmatrix}$$

Ma trận trực giao Q:

$$Q = \begin{pmatrix} 0.7303 & -0.0974 & -0.5943 & -0.3225 \\ 0.1826 & 0.9668 & 0.1316 & -0.1209 \\ 0.5477 & -0.2295 & 0.7909 & -0.1478 \\ 0.3651 & 0.0556 & -0.0635 & 0.9271 \end{pmatrix}$$

Ma trận tam giác trên R:

$$R = \begin{pmatrix} 5.4772 & 2.0083 & 6.2075 & 5.2947 \\ 0.0 & 4.7924 & -1.558 & 0.7025 \\ 0.0 & 0.0 & 2.8353 & 0.0804 \\ 0.0 & 0.0 & 0.0 & 5.4283 \end{pmatrix}$$

Ma trận tái tạo ($Q \times R$):

$$QR = \begin{pmatrix} 4.0 & 1.0 & 3.0 & 2.0 \\ 1.0 & 5.0 & 0.0 & 1.0 \\ 3.0 & 0.0 & 6.0 & 2.0 \\ 2.0 & 1.0 & 2.0 & 7.0 \end{pmatrix}$$

B.5.6. Đánh giá

Kết quả thực nghiệm cho thấy thuật toán không chỉ đảm bảo độ chính xác toán học tuyệt đối trong việc phân rã $A = QR$ mà còn xử lý ổn định quá trình trực giao hóa (Gram-Schmidt). Thuật toán thể hiện khả năng bao quát toàn bộ các kịch bản ma trận từ ma trận vuông đến ma trận hình chữ nhật thường gặp trong đại số tuyến tính thực hành, duy trì tốt tính trực chuẩn của ma trận Q và cấu trúc tam giác trên của ma trận R.

C Phân rã SVD (Singular Value Decomposition)

C.1 Cơ sở lý thuyết và Mục tiêu

Phân tích giá trị kỳ dị (Singular Value Decomposition) của ma trận A kích thước $m \times n$ thành tích của ba ma trận ($A = USV^T$). Trong đó, U là ma trận trực giao $m \times m$, V là ma trận trực giao $n \times n$, và S là ma trận đường chéo $m \times n$ chứa các giá trị kỳ dị.

Dạng tổng quát: $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$

- $\mathbf{U} \in \mathbb{R}^{m\times m}$: Ma trận trực giao (left singular vectors)
- $\Sigma \in \mathbb{R}^{m\times n}$: Ma trận đường chéo $\Sigma = \text{diag}(\sigma_1, \sigma_2, \cdots, \sigma_p)$, với σ_i là singular value của $\mathbf{A}$
- $\mathbf{V} \in \mathbb{R}^{n\times n}$: Ma trận trực giao (right singular vectors)

Liên hệ với chéo hóa: $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^T\mathbf{A})}$, và SVD tương đương chéo hóa của $\mathbf{A}^T\mathbf{A}$.

Tìm trị riêng:

Tìm các trị riêng của ma trận đối xứng M bằng thuật toán lặp QR. Phương pháp này liên tục phân tích và nhân ngược các ma trận Q, R để triệt tiêu các phần tử ngoài đường

chéo, giúp ma trận hội tụ về dạng đường chéo (hoặc tam giác trên) chứa các trị riêng. Hàm này cung cấp dữ liệu đầu vào thiết yếu cho hàm `svd(A)` để xác định các giá trị kỳ dị và cho hàm `eigen_vectors(M, lambdas)` để bắt đầu quá trình tìm không gian vector riêng.

Tìm và trực chuẩn hóa vector riêng:

Tìm không gian vector riêng tương ứng với các trị riêng và trực chuẩn hóa chúng để tạo thành một hệ cơ sở trực chuẩn. Hàm này được sử dụng bởi hàm `svd(A)` để xây dựng ma trận trực giao V và phối hợp với hàm `rank_and_basis(A)` để giải không gian nghiệm cho phương trình đặc trưng.

C.2 Thuật toán

C.2.1. Tìm trị riêng

- **Khởi tạo:** Gán $M_0 = M$.
- **Quá trình lặp:** Tại mỗi bước lặp $k = 0, 1, 2, \ldots$ (tối đa `max_iterations`):
  1. Phân tích QR ma trận hiện tại:
  $$M_k = Q_k R_k$$
  2. Cập nhật ma trận cho bước lặp tiếp theo bằng cách nhân ngược R_k và Q_k:
  $$M_{k+1} = R_k Q_k$$
- **Điều kiện hội tụ:** Thuật toán dừng lại khi tổng trị tuyệt đối của các phần tử nằm dưới đường chéo chính tiến cực kỳ gần về 0 (nhỏ hơn sai số ε):
$$\sum_i \sum_{j<i} |(M_k)_{ij}| < \varepsilon$$
- **Kết quả:** Khi ma trận hội tụ, các trị riêng λ_i chính là các phần tử trên đường chéo chính của M_k:
$$\lambda_i \approx (M_k)_{ii}$$
Trích xuất đường chéo này, sắp xếp theo thứ tự giảm dần và trả về.

C.2.2. Tìm và trực chuẩn hóa vector riêng

- **Xử lý trị riêng:** Lọc danh sách đầu vào `lambdas` để loại bỏ các giá trị trùng lặp (dựa trên sai số ε), thu được tập các trị riêng phân biệt λ_i.
- **Tìm và trực chuẩn hóa vector riêng:** Với mỗi trị riêng phân biệt λ_i:
  1. Lập ma trận đặc trưng bằng cách trừ đi λ_i trên đường chéo chính của M:
  $$A = M - \lambda_i I$$
  2. Tìm cơ sở cho không gian nghiệm (null space) của phương trình $Ax = 0$. Các vector cơ sở này chính là các vector riêng độc lập tuyến tính ứng với λ_i.
  3. Trực chuẩn hóa tập cơ sở vừa tìm được và thêm các vector kết quả vào danh sách V.
- **Bổ sung cơ sở (nếu thiếu):** Trong trường hợp ma trận chưa đủ n vector riêng (số chiều của V nhỏ hơn n):
  While $|V| < n$:
  - Tìm một vector mới v_{new} trực giao với tất cả các vector hiện có trong không gian V.
  - Chuẩn hóa v_{new} và thêm vào V. Dừng lại nếu không thể tìm thêm.

C.2.3. Phân rã SVD

- **Tìm ma trận V và S:**
  1. Tính ma trận đối xứng $M = A^T A$.
  2. Tìm các trị riêng của M, lọc các trị riêng dương $\lambda_i > \varepsilon$.
  3. Tính các giá trị kỳ dị σ_i:
  $$\sigma_i = \sqrt{\lambda_i}$$
  4. Tìm các vector riêng trực chuẩn v_i tương ứng với λ_i. Xây dựng ma trận V với các cột là v_i.
  5. Khởi tạo ma trận S kích thước $m \times n$ với các phần tử bằng 0. Gán các giá trị σ_i lên đường chéo chính: $S_{ii} = \sigma_i$.
- **Tìm ma trận U:**

1. Với mỗi giá trị kỳ dị $\sigma_i > \varepsilon$, tính vector u_i tương ứng:

$$u_i = \frac{Av_i}{\sigma_i}$$

2. Bổ sung các vector vào tập không gian (nếu số lượng vector tìm được $< m$) sao cho chúng tạo thành một cơ sở trực chuẩn của không gian $\mathbb{R}^m$.
3. Xây dựng ma trận U với các cột là các vector u_i đã trực chuẩn hóa.

C.3 Mã giả

Algorithm 10 Tìm trị riêng bằng lặp QR (`eigen_values`)

```
Input: Ma trận vuông M kích thước n × n
Output: Danh sách các trị riêng được sắp xếp giảm dần
max_iterations ← 1000
M_k ← bản sao của M
for iteration = 0 to max_iterations − 1 do
    Q, R ← qr_decomposition(M_k)
    M_k ← R × Q                                   ▷ Cập nhật lại M_{k+1}
    off_diag_sum ← Σ_i Σ_{j<i} |M_k[i][j]|
    if off_diag_sum < ε then                      ▷ Ma trận đã tiến đến dạng đường chéo
        break
    end if
end for
lambdas ← Danh sách các phần tử trên đường chéo chính của M_k
Sắp xếp lambdas theo thứ tự giảm dần
return lambdas
```

Algorithm 11 Tìm và trực chuẩn hóa vector riêng (`eigen_vectors`)

```
Input: Ma trận M (n × n), danh sách các trị riêng lambdas
Output: Ma trận V chứa các vector riêng trực chuẩn (mỗi vector là 1 dòng)
V ← ∅
unique_lambdas ← Lọc các giá trị không trùng lặp từ lambdas (sai số ε)
for mỗi λ_i ∈ unique_lambdas do
    A ← bản sao của M
    for j = 0 to n − 1 do
        A[j][j] ← A[j][j] − λ_i                    ▷ Tạo ma trận M − λ_i I
    end for
    _, _, _, null_basis ← rank_and_basis(A)        ▷ Tìm không gian nghiệm
    ortho_basis ← orthoNormalize(null_basis)
    for mỗi v ∈ ortho_basis do
        Thêm v vào V
    end for
end for
while |V| < n do                ▷ Bổ sung cơ sở trực chuẩn nếu chưa đủ chiều
    Tìm vector new_v trực giao với tất cả các vector đang có trong V
    if Tìm thấy new_v then
        Chuẩn hóa và thêm new_v vào V
    else
        break
    end if
end while
return V
```

Algorithm 12 Phân tích SVD (`svd`)

Input: Ma trận A kích thước $m \times n$
Output: Các ma trận U, S, V sao cho $A = USV^T$
$M \leftarrow A^T \times A$ ▷ Tính ma trận đối xứng
lambdas $\leftarrow$ `eigen_values`(M) ▷ Tìm trị riêng của M
lambdas_pos $\leftarrow$ Lọc các giá trị $\lambda > \varepsilon$ từ lambdas
sigmas $\leftarrow [\sqrt{\lambda}$ for $\lambda \in$ lambdas_pos$]$ ▷ Tính giá trị kỳ dị
eigvecs $\leftarrow$ `eigen_vectors`$(M,$ lambdas_pos$)$ ▷ Tìm các vector riêng
$V \leftarrow$ `build_V`(eigvecs) ▷ Tạo ma trận V
$S \leftarrow$ `build_S`$(A,$ sigmas$)$ ▷ Tạo ma trận đường chéo S kích thước $m \times n$
Xây dựng ma trận U (`build_U`)**:**
$U \leftarrow \emptyset$
for $i = 0$ **to** $\min(m,n) - 1$ **do**
    $\sigma_i \leftarrow S[i][i]$
    if $\sigma_i > \varepsilon$ **then**
        $v_i \leftarrow$ cột i của V
        $u_i \leftarrow (A \times v_i)/\sigma_i$
        Thêm cột u_i vào dạng dòng trong danh sách U
    end if
end for
while $|U| < m$ **do** ▷ Bổ sung cơ sở trực chuẩn cho đủ không gian $\mathbb{R}^m$
    Trực chuẩn hóa U
    Tìm vector new_u trực giao với các vector trong U
    Thêm new_u vào U nếu tìm thấy
end while
$U \leftarrow U^T$ ▷ Chuyển vị để tạo ma trận U với các vector ở dạng cột
return U, S, V

C.4 Cài đặt và các hàm chính

```python
def multiply(A: list[list], B: list[list]) -> list[list]:
    """Tra ve tich cua A va B"""
    if len(A)*len(B) == 0 or len(A[0])*len(B[0]) == 0 or len(A[0])
    != len(B):
        return []

    m, n = len(A), len(B[0])

    # Khoi tao ma tran ket qua voi kich thuoc m x n
    result = [[0] * n for _ in range(m)]
    # Tinh tich A * B
    for i in range(m):
        for j in range(n):
            result[i][j] = sum(A[i][k] * B[k][j] for k in range(
```

```
len(B)))

    return result

def normalizevector(v: list) -> list:
    """Tra ve vector v da duoc chuan hoa (||v|| = 1)"""
    # Do dai n cua v
    n = sum(x**2 for x in v) ** 0.5
    if n == 0:
        return v

    # Chia tung phan tu roi tra ve
    return [x / n for x in v]

# M: tranpose(A)*A
def eigen_values(M: list[list]) -> list:
    # Tinh tong duong cheo
    def off_diagonal_sum(A):
        return sum(abs(A[i][j]) for i in range(len(A)) for j in
range(i))
    # Lap QR
    max_iterations = 1000
    Mk = [row[:] for row in M] # Copy lai ma tran goc

    for iteration in range(max_iterations):
        # Phan tich QR
        Q, R = qr_decomposition(Mk)
        # Cap nhat Mk+1 = R * Q
        Mk = multiply(R, Q)
        if off_diagonal_sum(Mk) < epsilon:
            break

    return sorted([Mk[i][i] for i in range(len(Mk))], reverse =
True)

def orthoNormalize(vectors: list[list]) -> list[list]:
    """Tra ve danh sach cac vector da duoc truc chuan hoa (
orthonormalized)"""
    # Su dung thuat toan Gram-Schmidt de truc chuan hoa
    orthonormalized = []
    for v in vectors:
        w = copy.deepcopy(v)
        for u in orthonormalized:
            l_u = sum(u[i] * u[i] for i in range(len(u)))  # ||u
```

```
||^2
            if l_u == 0:
                continue

            proj = sum(w[i] * u[i] for i in range(len(w))) / l_u # (w . u) / ||u||^2
            w = [w[i] - proj * u[i] for i in range(len(w))] # w - proj_u(w)
        orthonormalized.append(normalizevector(w))
    return orthonormalized

def eigen_vectors(M: list[list], lambdas: list) -> list[list]:
    n = len(M)
    V = []
    unique_lambdas = [] # Tranh tinh toan lai cho cac gia tri trung lap
    for lam in lambdas:
        if not any(abs(lam - u_lam) < epsilon for u_lam in unique_lambdas):
            unique_lambdas.append(lam)

    for lambda_i in unique_lambdas:
        A = copy.deepcopy(M)
        for i in range(n):
            A[i][i] -= lambda_i

        _, __, ___, null_basis = rank_and_basis(A)

        # Truc chuan hoa tung vector trong null_basis va them vao V
        ortho_basis = orthoNormalize(null_basis)
        for v in ortho_basis:
            V.append(v)

    while len(V) < n:
        new_v = None
        for t in range(n):
            w = [0.0] * n
            w[t] = 1.0
            for u in V:
                proj = sum(w[p] * u[p] for p in range(n))
                w = [w[p] - proj * u[p] for p in range(n)]
            nw = sum(x*x for x in w) ** 0.5
            if nw > epsilon:
```

```
                new_v = [x / nw for x in w]
                break
        if new_v is not None:
            V.append(new_v)
        else:
            break
    return V

def build_V(v: list) -> list[list]:
    return transpose(v)

def build_S(A: list[list], sigmas: list) -> list[list]: # SIGMA
    """Tra ve ma tran S (ma tran duong cheo) chua cac o_i tren duong cheo chinh"""
    m = len(A)
    n = len(A[0])

    S = [[0.0] * n for _ in range(m)] # S_mxn

    for i in range(min(m, n, len(sigmas))):
        S[i][i] = sigmas[i]

    return S

def build_U(A: list[list], V: list[list], S: list[list]) -> list[list]:
    U = []
    m = len(A)
    n = len(A[0])
    num_sigmas = min(m, n)

    for i in range(num_sigmas):
        sigma_i = S[i][i]
        if sigma_i > epsilon:
            v_i = [[row[i]] for row in V]
            Av_i = multiply(A, v_i)
            u_i = [[Av_i[j][0] / sigma_i] for j in range(len(Av_i))]
            U.append(transpose(u_i)[0])

    # Bo sung them cac vector truc chuan vao U neu thieu
    while len(U) < m:
        ortho = [normalizevector(u) for u in U]
        new_u = None
```

```
        for t in range(m):
            w = [0.0] * m
            w[t] = 1.0
            for u in ortho:
                proj = sum(w[p] * u[p] for p in range(m))
                w = [w[p] - proj * u[p] for p in range(m)]
            nw = sum(x*x for x in w) ** 0.5
            if nw > epsilon:
                new_u = [x / nw for x in w]
                break
        if new_u is not None:
            U.append(new_u)
        else:
            break

    return transpose(U)

def svd(A: list[list]):
    M = multiply(transpose(A), A)

    lambdas = eigen_values(M)
    lambdas_pos = [l for l in lambdas if l > epsilon]
    sigmas = [l**0.5 for l in lambdas_pos]
    eigvecs = eigen_vectors(M, lambdas_pos)

    V = build_V(eigvecs)
    S = build_S(A, sigmas)
    U = build_U(A, V, S)
    return U, S, V
```

C.5 Kết quả thực nghiệm (Test Cases) và Đánh giá

Thuật toán phân rã $A = USV^T$ đã được cài đặt thực nghiệm bằng ngôn ngữ Python và được đối chiếu độ chính xác với hàm `numpy.linalg.svd` của thư viện `NumPy` chuyên dụng. Các kịch bản kiểm thử (Test Cases) được thiết kế bao quát các dạng ma trận khác nhau để kiểm chứng tính đúng đắn và độ ổn định của thuật toán. Tất cả các trường hợp đều thỏa mãn hằng đẳng thức $A = U \cdot S \cdot V^T$, với U, V là các ma trận trực giao và S là ma trận đường chéo chứa các giá trị kỳ dị (singular values) dương giảm dần.

C.5.1. Test Case 1: Ma trận vuông 2×2.

$$A_1 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Kết quả phân tích $A_1 = USV^T$:

$$U = \begin{pmatrix} 0.4046 & 0.9145 \\ 0.9145 & -0.4046 \end{pmatrix}, \quad S = \begin{pmatrix} 5.4650 & 0.0 \\ 0.0 & 0.3660 \end{pmatrix}, \quad V = \begin{pmatrix} 0.5760 & -0.8174 \\ 0.8174 & 0.5760 \end{pmatrix}$$

C.5.2. Test Case 2: Ma trận suy biến (Rank 1).

$$A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$$

Kết quả:

$$U = \begin{pmatrix} 0.4472 & 0.8944 \\ 0.8944 & -0.4472 \end{pmatrix}, \quad S = \begin{pmatrix} 5.0 & 0.0 \\ 0.0 & 0.0 \end{pmatrix}, \quad V = \begin{pmatrix} 0.4472 & 0.8944 \\ 0.8944 & -0.4472 \end{pmatrix}$$

C.5.3. Test Case 3: Ma trận trực giao.

$$A_3 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

Kết quả:

$$U = \begin{pmatrix} 0.0 & -1.0 \\ 1.0 & 0.0 \end{pmatrix}, \quad S = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}, \quad V = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}$$

C.5.4. Test Case 4: Ma trận vector cột (3×1).

$$A_4 = \begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix}$$

Kết quả:

$$U = \begin{pmatrix} 0.6 & 0.8 & 0.0 \\ 0.8 & -0.6 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{pmatrix}, \quad S = \begin{pmatrix} 5.0 \\ 0.0 \\ 0.0 \end{pmatrix}, \quad V = \begin{pmatrix} 1.0 \end{pmatrix}$$

C.5.5. Test Case 5: Ma trận Zero 2×2.

$$A_5 = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Kết quả:

$$U = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}, \quad S = \begin{pmatrix} 0.0 & 0.0 \\ 0.0 & 0.0 \end{pmatrix}, \quad V = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}$$

C.5.6. Test Case 6: Ma trận vuông 4×4 đối xứng.

$$A_6 = \begin{pmatrix} 4 & 1 & 3 & 2 \\ 1 & 5 & 0 & 1 \\ 3 & 0 & 6 & 2 \\ 2 & 1 & 2 & 7 \end{pmatrix}$$

Kết quả:

$$U = \begin{pmatrix} 0.4790 & -0.1275 & -0.2970 & -0.8161 \\ 0.1959 & 0.7347 & -0.6103 & 0.2223 \\ 0.5784 & -0.5513 & -0.2851 & 0.5294 \\ 0.6306 & 0.3744 & 0.6767 & 0.0653 \end{pmatrix}$$

$$S = \begin{pmatrix} 10.6645 & 0 & 0 & 0 \\ 0 & 5.3359 & 0 & 0 \\ 0 & 0 & 4.3778 & 0 \\ 0 & 0 & 0 & 1.6217 \end{pmatrix}, \quad V = U \text{ (do } A_6 \text{ đối xứng)}$$

C.5.7. Đánh giá

Kết quả thực nghiệm cho thấy thuật toán đảm bảo độ chính xác toán học cao trong việc trích xuất các giá trị kỳ dị (singular values) và các vector trực giao. Thuật toán xử lý an

toàn các ma trận suy biến hoặc gần suy biến, phản ánh đúng các tính chất hình học phức tạp và bao quát toàn bộ các kịch bản ma trận thực (cả vuông và chữ nhật) thường gặp trong ứng dụng phân tích dữ liệu và đại số tuyến tính.

D Chéo hóa ma trận

D.1 Cơ sở lý thuyết và Mục tiêu

D.1.1. Đặt vấn đề

Cho một ma trận vuông $A \in \mathbb{R}^{n\times n}$. Chéo hóa ma trận là quá trình tìm ma trận đường chéo D và ma trận khả nghịch P sao cho:

$$A = PDP^{-1}$$

Trong đó các cột của P là các vector riêng của A, còn các phần tử trên đường chéo chính của D là các trị riêng tương ứng.

Một ma trận $A \in \mathbb{R}^{n\times n}$ **chéo hóa được** khi và chỉ khi nó có đúng n vector riêng độc lập tuyến tính. Điều này tương đương với việc với mỗi trị riêng λ, số chiều của không gian riêng $\ker(A - \lambda I)$ bằng đúng bậc đại số của λ.

Thuật toán được trình bày dưới đây không phụ thuộc vào thư viện đại số tuyến tính bên ngoài cho các bước tính vector riêng, nghịch đảo và nhân ma trận — ngoại trừ việc dùng `numpy.linalg.eigvals` để tính trị riêng.

D.1.2. Cấu trúc ma trận thành phần

Sau khi thực hiện chéo hóa thành công, ta thu được ba ma trận:

- **Ma trận chuyển cơ sở $P \in \mathbb{R}^{n\times n}$:** Mỗi *cột* của P là một vector riêng của A. Ma trận P là khả nghịch khi và chỉ khi các vector riêng này độc lập tuyến tính. Các vector riêng không được chuẩn hóa — biến tự do được gán bằng 1.
- **Ma trận đường chéo $D \in \mathbb{R}^{n\times n}$:** Là ma trận đường chéo với các trị riêng λ_i nằm trên đường chéo chính ($d_{i,i} = \lambda_i$), sắp xếp theo thứ tự tương ứng với các cột của P.
- **Ma trận nghịch đảo $P^{-1} \in \mathbb{R}^{n\times n}$:** Được tính bằng phương pháp Gauss-Jordan trên ma trận tăng cường $[P \mid I]$.

D.2 Thuật toán

Bước D.2.1: Tính trị riêng

Dùng `numpy.linalg.eigvals` để tính tất cả n trị riêng (kể cả bội số) của A. Nếu bất kỳ trị riêng nào có phần ảo $|\mathrm{Im}(\lambda)| > 10^{-6}$, thuật toán từ chối và trả về `None` vì chỉ hỗ trợ ma trận thực với trị riêng thực.

Bước D.2.2: Tìm vector riêng (Null Space)

Với mỗi trị riêng phân biệt λ, xây dựng ma trận:

$$M = A - \lambda I$$

Sau đó tìm không gian null $\ker(M)$ bằng cách:

1. Đưa M về dạng bậc thang (REF) bằng khử Gauss với chọn phần tử trội từng phần.
2. Xác định các cột pivot và cột tự do.
3. Với mỗi biến tự do, gán giá trị 1 và dùng thế ngược (back substitution) để tính các biến pivot tương ứng.

Mỗi cột tự do sinh ra một vector riêng độc lập tuyến tính.

Bước D.2.3: Kiểm tra điều kiện chéo hóa

Ghép tất cả vector riêng tìm được thành ma trận P (mỗi vector là một cột). Tính rank của P bằng khử Gauss. Ma trận A chéo hóa được khi và chỉ khi:

$$\mathrm{rank}(P) = n$$

tức là có đúng n vector riêng độc lập tuyến tính.

Bước D.2.4: Tính P^{-1} bằng Gauss-Jordan

Xây dựng ma trận tăng cường $[P \mid I_n]$ và thực hiện khử Gauss-Jordan đầy đủ (cả lên và xuống). Phần bên phải thu được chính là P^{-1}.

D.3 Mã giả

Algorithm 13 Chéo hóa ma trận $A = PDP^{-1}$

Require: Ma trận vuông $A \in \mathbb{R}^{n\times n}$ với trị riêng thực
Ensure: Ma trận P, ma trận đường chéo D, và P^{-1}; hoặc `None` nếu không chéo hóa được

```
eigenvalues ← numpy.eigvals(A)
if ∃λ ∈ eigenvalues sao cho |Im(λ)| > 10^-6 then
    return None, None, None
end if
unique_eigs ← các trị riêng phân biệt (phần thực)
all_eigenvectors ← [ ], matched_eigenvals ← [ ]
for mỗi λ ∈ unique_eigs do
    M ← A − λI
    (REF, pivot_cols) ← row_echelon_form(M)
    null_vecs ← back_substitution_null_space(REF, pivot_cols)
    Thêm null_vecs vào all_eigenvectors
    Thêm λ (với số lần bằng |null_vecs|) vào matched_eigenvals
end for
P ← ma trận ghép từ all_eigenvectors (mỗi vector là một cột)
if rank(P) ≠ n then
    return None, None, None                ▷ Không đủ vector riêng độc lập
end if
D ← diag(matched_eigenvals)
P^-1 ← gauss_jordan_inverse(P)
return P, D, P^-1
```

Algorithm 14 Tìm không gian null qua REF và thế ngược

Require: Ma trận $M \in \mathbb{R}^{n\times n}$, danh sách cột pivot
Ensure: Danh sách các vector cơ sở của $\ker(M)$

```
free_cols ← {0, ..., n−1} \ pivot_cols
null_vectors ← [ ]
for mỗi cột tự do fc ∈ free_cols do
    Khởi tạo v ← 0_n; đặt v[fc] ← 1
    for i = |pivot_cols| − 1 downto 0 do
        pc ← pivot_cols[i]
        v[pc] ← −(1/REF[i, pc]) Σ_{c≠pc} REF[i, c] · v[c]
    end for
    Thêm v vào null_vectors
end for
return null_vectors
```

D.4 Cài đặt và các hàm chính

```
import numpy as np  # Chi dung de tinh tri rieng (eigvals)

# PHAN 1: CAC HAM TIEN ICH CO BAN
def copy_matrix(M):
    return [row[:] for row in M]

def mat_mul(A, B):
    n = len(A)
    m = len(A[0])
    p = len(B[0])
    return [[sum(A[i][k] * B[k][j] for k in range(m)) for j in
   range(p)] for i in range(n)]

# PHAN 2: KHU GAUSS (REF) VOI PARTIAL PIVOTING
def row_echelon_form(M):
    M = copy_matrix(M)
    rows, cols = len(M), len(M[0])
    pivot_cols = []
    row_idx = 0
    row_swaps = 0
    EPS = 1e-9

    for col in range(cols):
        # Partial pivoting
        max_val = EPS
        max_row = None
        for r in range(row_idx, rows):
            if abs(M[r][col]) > max_val:
                max_val = abs(M[r][col])
                max_row = r

        if max_row is None:
            continue

        if max_row != row_idx:
            M[row_idx], M[max_row] = M[max_row], M[row_idx]
            row_swaps += 1

        pivot_cols.append(col)

        for r in range(row_idx + 1, rows):
            if abs(M[r][col]) > EPS:
```

```
                factor = M[r][col] / M[row_idx][col]
                for c in range(col, cols):
                    M[r][c] -= factor * M[row_idx][c]
                M[r][col] = 0.0

        row_idx += 1

    return M, pivot_cols, row_swaps

# PHAN 3: THE NGUOC (BACK SUBSTITUTION) TIM NULL SPACE
def back_substitution_null_space(ref, pivot_cols, num_cols):
    EPS = 1e-9
    num_rows = len(ref)
    free_cols = [c for c in range(num_cols) if c not in pivot_cols
   ]

    null_vectors = []
    for fc in free_cols:
        vec = [0.0] * num_cols
        vec[fc] = 1.0

        for i in range(len(pivot_cols) - 1, -1, -1):
            pc = pivot_cols[i]
            if i >= num_rows:
                continue
            total = 0.0
            for c in range(pc + 1, num_cols):
                if abs(ref[i][c]) > EPS:
                    total += ref[i][c] * vec[c]
            vec[pc] = -total / ref[i][pc]

        null_vectors.append(vec)

    return null_vectors

# PHAN 4: TIM VECTOR RIENG
def build_A_minus_lambdaI(A, lam):
    n = len(A)
    return [
        [A[i][j] - (lam if i == j else 0.0) for j in range(n)]
        for i in range(n)
    ]

def find_eigenvectors(A, eigenvalues):
```

```
    n = len(A)

    # Lay tri rieng dac biet (phan thuc)
    unique_eigs = []
    seen = []
    for lam in eigenvalues:
        lam_real = lam.real
        if not any(abs(lam_real - s) < 1e-6 for s in seen):
            unique_eigs.append(lam_real)
            seen.append(lam_real)

    all_eigenvectors = []
    matched_eigenvals = []

    for lam in unique_eigs:
        M = build_A_minus_lambdaI(A, lam)
        ref, pcols, _ = row_echelon_form(M)
        null_vecs = back_substitution_null_space(ref, pcols, n)

        # KHONG CHUAN HOA: dung truc tiep vector tho
        all_eigenvectors.extend(null_vecs)
        matched_eigenvals.extend([lam] * len(null_vecs))

    return all_eigenvectors, matched_eigenvals

# PHAN 5: KIEM TRA CHEO HOA DUOC
def matrix_rank(M):
    """Tinh rank bang so pivot sau khi khu Gauss."""
    _, pivot_cols, _ = row_echelon_form(M)
    return len(pivot_cols)

def is_diagonalizable(A, eigenvectors):
    """Kiem tra dieu kien cheo hoa: co dung n vector rieng doc lap
    tuyen tinh."""
    n = len(A)
    if len(eigenvectors) < n:
        return False
    P = build_P(eigenvectors)
    return matrix_rank(P) == n

# PHAN 6:XAY DUNG MA TRAN P VA D
def build_P(eigenvectors):
    """Ma tran P: moi COT la mot vector rieng."""
    n = len(eigenvectors[0])
```

```
    return [[eigenvectors[j][i] for j in range(n)] for i in range(n)]

def build_D(matched_eigenvals):
    """Ma tran D: tri rieng nam tren duong cheo chinh."""
    n = len(matched_eigenvals)
    return [[float(matched_eigenvals[i]) if i == j else 0.0 for j in range(n)] for i in range(n)]

# PHAN 7: NGHICH DAO MA TRAN P (GAUSS-JORDAN)
def mat_inverse(M):
    """Tinh nghich dao bang Gauss-Jordan."""
    n = len(M)
    EPS = 1e-9

    aug = [M[i][:] + [1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]

    for col in range(n):
        max_row = max(range(col, n), key=lambda r: abs(aug[r][col]))
        if abs(aug[max_row][col]) < EPS:
            raise ValueError(f"Ma tran suy bien tai cot {col}, khong co nghich dao!")

        aug[col], aug[max_row] = aug[max_row], aug[col]

        pivot = aug[col][col]
        aug[col] = [x / pivot for x in aug[col]]

        for r in range(n):
            if r != col and abs(aug[r][col]) > EPS:
                factor = aug[r][col]
                aug[r] = [aug[r][c] - factor * aug[col][c] for c in range(2 * n)]

    return [aug[i][n:] for i in range(n)]

# PHAN 8: HAM CHINH - CHEO HOA MA TRAN
def diagonalize(A):
    """
    Cheo hoa ma tran A = P D P_inv.

    - Vector rieng KHONG duoc chuan hoa (free variable = 1).
```

```
    - Chi ho tro ma tran vuong thuc voi tri rieng thuc.
    - Tra ve (P, D, P_inv) neu cheo hoa duoc,
      nguoc lai tra ve (None, None, None).
    """

    n = len(A)
    # Buoc 1: Tinh tri rieng
    eigenvalues = np.linalg.eigvals(A)

    # Tu choi neu co tri rieng phuc thuc su
    if any(abs(lam.imag) > 1e-6 for lam in eigenvalues):
        return None, None, None

    # Buoc 2: Tim vector rieng (KHONG chuan hoa)
    eigenvectors, matched_eigenvals = find_eigenvectors(A,
  eigenvalues)

    # Buoc 3: Kiem tra cheo hoa duoc
    if not is_diagonalizable(A, eigenvectors):
        return None, None, None

    # Buoc 4: Xay dung P, D, P^-1
    P = build_P(eigenvectors)
    D = build_D(matched_eigenvals)
    P_inv = mat_inverse(P)

    return P, D, P_inv
```

D.5 Kết quả thực nghiệm (Test Cases) và Đánh giá

Thuật toán chéo hóa ma trận $A = PDP^{-1}$ đã được cài đặt thực nghiệm bằng ngôn ngữ Python và được đối chiếu độ chính xác thông qua hàm `numpy.linalg.eig` của thư viện `NumPy` chuyên dụng. Các kịch bản kiểm thử (Test Cases) được thiết kế bao quát các dạng ma trận khả nghịch và không suy biến khác nhau để kiểm chứng tính đúng đắn của thuật toán. Tất cả các trường hợp đều thỏa mãn hằng đẳng thức $A = P \cdot D \cdot P^{-1}$ (hoặc tương đương $A \cdot P = P \cdot D$), trong đó D là ma trận đường chéo chứa các giá trị riêng và P là ma trận chứa các vector riêng độc lập tuyến tính tương ứng.

D.5.1. Ma trận chéo hóa được với trị riêng phân biệt

Đây là trường hợp cơ bản và thuận lợi nhất. Khi A có n trị riêng phân biệt, mỗi trị riêng

tự động sinh ra đúng một vector riêng độc lập, đảm bảo điều kiện chéo hóa.

Test Case 1: Ma trận 3×3 với ba trị riêng phân biệt

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ 0 & 1 & 4 \end{bmatrix} \quad \Rightarrow \quad \lambda_1 = 2,\ \lambda_2 = 3,\ \lambda_3 = 4$$

$$P = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 1 & -1 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

D.5.2. Ma trận chéo hóa được với trị riêng bội

Ma trận có trị riêng bội vẫn có thể chéo hóa được nếu bậc hình học (số chiều không gian riêng) bằng bậc đại số. Thuật toán tìm null space qua biến tự do xử lý chính xác trường hợp này.

Test Case 2: Ma trận 3×3 với trị riêng bội 2

$$A = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad \Rightarrow \quad \lambda_{1,2} = 3 \text{ (bội 2)},\ \lambda_3 = 5$$

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

Trị riêng $\lambda = 3$ có bậc hình học $= 2$ (hai biến tự do), tương ứng bậc đại số $= 2$, nên chéo hóa được.

D.5.3. Các trường hợp đặc biệt (Edge Cases)

Thuật toán xử lý đúng đắn các tình huống biên quan trọng:

- **Ma trận không chéo hóa được (Defective Matrix):** Khi trị riêng có bậc hình học nhỏ hơn bậc đại số, ma trận P không có rank đầy đủ. Thuật toán phát hiện điều này và trả về `None`.

Test Case 3: Ma trận Jordan 2×2 không chéo hóa được

$$A = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix} \quad \Rightarrow \quad \lambda_{1,2} = 3 \text{ (bội 2)}, \quad \ker(A - 3I) = \operatorname{span}\left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\}$$

Bậc hình học $= 1 < 2 =$ bậc đại số $\Rightarrow$ **Không chéo hóa được.** Thuật toán trả về `None`.

- **Ma trận với trị riêng phức:** Thuật toán kiểm tra ngay sau khi tính trị riêng. Nếu phát hiện $|\mathrm{Im}(\lambda)| > 10^{-6}$, thuật toán từ chối xử lý và trả về `None`.

  Test Case 4: Ma trận xoay có trị riêng phức

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \quad \Rightarrow \quad \lambda_{1,2} = \pm i$$

  Vì trị riêng là phức thuần ảo $\Rightarrow$ **Từ chối.** Thuật toán trả về `None`.

- **Ma trận đường chéo:** Ma trận đường chéo luôn chéo hóa được (trường hợp tầm thường). Các vector riêng là các vector đơn vị chuẩn $\mathbf{e}_i$ và $P = I$, $D = A$.

  Test Case 5: Ma trận đường chéo 3×3

$$A = \begin{bmatrix} 4 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 7 \end{bmatrix} \quad \Rightarrow \quad P = I_3, \quad D = A$$

- **Ma trận đối xứng:** Theo định lý phổ (Spectral Theorem), ma trận đối xứng thực luôn chéo hóa được với n vector riêng trực giao nhau. Thuật toán xác nhận điều kiện chéo hóa chính xác, tuy nhiên không chuẩn hóa các vector riêng.

  Test Case 6: Ma trận đối xứng 2×2

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix} \quad \Rightarrow \quad \lambda_1 = 3,\ \lambda_2 = 5, \quad P = \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$$

D.5.4. Đánh giá

Kết quả thực nghiệm cho thấy thuật toán đảm bảo độ chính xác toán học, phát hiện đúng các trường hợp không chéo hóa được, và bao quát toàn bộ các kịch bản ma trận vuông thực thường gặp trong đại số tuyến tính.

E Trực quan hóa bằng Manim

Để làm rõ bản chất toán học và minh họa sinh động các bước tính toán của các thuật toán phân rã ma trận, nhóm đã sử dụng thư viện **Manim Community (v0.18+)** để xây dựng các video hoạt ảnh (animations). Các kịch bản trực quan hóa được chia thành 4 chủ đề chính, tương ứng với các kỹ thuật đại số tuyến tính cốt lõi trong đồ án.

E.1 Phân rã LU với kỹ thuật Partial Pivoting

Kịch bản này mô phỏng chi tiết quá trình biến đổi ma trận A thành $P \cdot A = L \cdot U$.

- **Khởi tạo:** Video bắt đầu bằng việc hiển thị hệ thống các ma trận khởi tạo: $U = A$, $L = I$, và $P = I$.
- **Hoạt ảnh Pivoting:** Tại mỗi bước khử cột, Manim sử dụng ‘SurroundingRectangle‘ để làm nổi bật cột đang xét. Khi tìm thấy phần tử trội, hệ thống tự động sinh hoạt ảnh di chuyển (hoán vị) toàn bộ dữ liệu của hai dòng tương ứng trên cả ma trận U và ma trận hoán vị P, đồng thời đổi chỗ các phần tử tương ứng đã tính trước đó trong L.
- **Tính toán trực quan:** Các phép tính tìm hệ số khử l_{ij} và phép biến đổi dòng ($R_i \leftarrow R_i - l_{ij}R_j$) được hiển thị rõ ràng trên màn hình trước khi cập nhật vào ma trận.
- **Kiểm chứng:** Video kết thúc bằng việc so sánh trực tiếp kết quả phép nhân $P \cdot A$ và $L \cdot U$ để khẳng định tính đúng đắn của thuật toán.

E.2 Phân rã QR bằng phương pháp Gram-Schmidt

Kịch bản này diễn giải quá trình xây dựng ma trận trực giao Q và ma trận tam giác trên R.

- **Trực giao hóa:** Tách ma trận A thành các vector cột u_i. Áp dụng công thức Gram-Schmidt để sinh ra các vector trực giao v_i. Mỗi bước đều có sự biến đổi (Transform) nhịp nhàng từ công thức sang con số cụ thể.
- **Chuẩn hóa:** Xây dựng các vector trực chuẩn q_i từ v_i và ghép chúng lại thành ma trận Q hoàn chỉnh.

- **Xây dựng R:** Ma trận R được tính thông qua tích vô hướng $R_{ij} = \langle q_i, u_j \rangle$. Video sử dụng kỹ thuật highlight từng ô (cell) của ma trận R kết hợp với công thức tính chi tiết đang chạy phía dưới để người xem dễ dàng đối chiếu.

E.3 Phân rã SVD và Ý nghĩa hình học

Đây là kịch bản mang tính hình học cao nhất, giúp giải thích ý nghĩa thực sự của phép phân rã Singular Value Decomposition ($A = U\Sigma V^T$).

- **Thiết lập không gian:** Sử dụng đối tượng ‘NumberPlane‘ cùng một đường tròn đơn vị (Unit Circle) và hai vector cơ sở $\hat{i}, \hat{j}$ tại vị trí tâm.
- **Ba bước biến đổi tuyến tính:** Phép nhân ma trận A được tách thành 3 hoạt ảnh liên tiếp áp dụng hàm ‘applymatrix‘:
  1. Nhân với V^T: Minh họa phép **Xoay (Rotation)** hệ trục.
  2. Nhân với Σ: Minh họa phép **Co giãn (Scaling)** biến đường tròn thành hình ellipse.
  3. Nhân với U: Minh họa phép **Xoay (Rotation)** lần cuối để đạt được vị trí của không gian ma trận A.
- Nhóm áp dụng tính năng ‘addupdater‘ của Manim để các nhãn vector (labels) luôn bám sát theo sự chuyển động của các vector cơ sở trong không gian thực.

E.4 Chéo hóa Ma trận $A = PDP^{-1}$

Kịch bản mô tả trọn vẹn đường ống (pipeline) để chéo hóa một ma trận vuông.

- **Tìm giá trị riêng (Eigenvalues):** Trình bày từng bước giải phương trình đa thức đặc trưng $\det(A - \lambda I) = 0$ để tìm ra λ_1, λ_2.
- **Tìm vector riêng (Eigenvectors):** Áp dụng phép khử Gauss trên $(A - \lambda I)$. Nhóm đã sử dụng màu sắc (Tím/Cam) để phân biệt rõ ràng "biến chốt" (pivot) và "biến tự do" (free).
- **Tối ưu hiển thị:** Nhóm đã chủ đích loại bỏ việc chuẩn hóa các vector riêng (loại bỏ các số vô tỉ như $1/\sqrt{2}$) bằng cách gán thẳng biến tự do bằng 1. Điều này giúp các ma trận P, D, P^{-1} chứa các số nguyên gọn gàng, giúp người xem dễ theo dõi hơn.

E.5 Ứng dụng SVD: Nén ảnh (Image Compression)

Kịch bản này trực quan hóa ứng dụng thực tiễn của phân rã SVD trong việc nén dữ liệu ảnh thông qua kỹ thuật giữ lại k giá trị kỳ dị lớn nhất (Truncated SVD).

- **Trực quan hóa ma trận bằng khối hình:** Thay vì hiển thị các con số phức tạp, nhóm sử dụng các khối hình chữ nhật ('Rectangle') với diện tích tương ứng để biểu diễn kích thước của các ma trận $A_{m\times n}$, $U_{m\times m}$, $\Sigma_{m\times n}$ và $V^T_{n\times n}$. Từ đó, người xem dễ dàng hình dung dung lượng lưu trữ gốc của bức ảnh là $m \times n$.
- **Mô phỏng quá trình cắt ma trận (Truncated SVD):** Manim sử dụng các nét đứt ('DashedLine') để mô phỏng nhát cắt trên các ma trận U, Σ, V^T. Phần dữ liệu bị loại bỏ sẽ được làm mờ đi (giảm opacity), làm nổi bật phần ma trận lõi U_k, Σ_k, V_k^T được giữ lại và ghép sát vào nhau.
- **So sánh hiệu năng lưu trữ:** Video kết thúc bằng việc đối chiếu dung lượng lưu trữ mới $k(m+n+1)$ vô cùng nhỏ gọn so với ban đầu. Đồng thời, dấu bằng ($=$) được hiệu ứng chuyển hóa (Transform) thành dấu xấp xỉ ($\approx$) để nhấn mạnh bản chất của việc nén ảnh là có mất mát thông tin ($A \approx U_k \Sigma_k V_k^T$).

E.6 Ứng dụng LU: Giải hệ phương trình tuyến tính

Kịch bản này làm rõ quy trình và ưu điểm của việc sử dụng ma trận phân rã LU để giải hệ phương trình $A\mathbf{x} = \mathbf{b}$, đặc biệt tối ưu khi vector đầu vào $\mathbf{b}$ thay đổi liên tục.

- **Trình bày ý tưởng thuật toán:** Video mở đầu bằng việc tách cấu trúc $A\mathbf{x} = \mathbf{b}$ thành $LU\mathbf{x} = \mathbf{b}$. Nhóm sử dụng khung viền highlight ('SurroundingRectangle') và sự đồng bộ về màu sắc (Xanh lá/Xanh dương) để phân tách rõ ràng bài toán thành hai giai đoạn: giải hệ tam giác dưới $L\mathbf{y} = \mathbf{b}$ (thế tiến) và giải hệ tam giác trên $U\mathbf{x} = \mathbf{y}$ (thế lùi).
- **Minh họa bằng số liệu cụ thể (Hệ 3x3):** Hệ thống lần lượt tính toán và hiển thị ma trận kết quả trung gian $\mathbf{y}$, sau đó chuyển tiếp mượt mà để sử dụng vector $\mathbf{y}$ này tìm ra nghiệm cuối cùng $\mathbf{x}$. Các con số được hiển thị theo từng nấc giúp người xem dễ dàng theo dõi quy trình thế nghiệm.
- **Kết luận tối ưu hóa:** Thông điệp cuối cùng của hoạt ảnh là lời khẳng định về độ phức tạp tính toán được giảm đáng kể từ $O(n^3)$ xuống $O(n^2)$ ở bước giải hệ, làm nổi

bật giá trị thực tiễn và tính ứng dụng cao của phương pháp phân rã LU trong tính toán khoa học.

E.7 Truy cập video Trực quan hóa

Để quý thầy cô và người đọc có thể trực tiếp trải nghiệm và đánh giá các hoạt ảnh trực quan hóa đã được trình bày ở các mục trên, nhóm đã xuất bản (render) và lưu trữ toàn bộ các video hoạt ảnh lên các nền tảng trực tuyến.

Để tiện cho việc theo dõi các chủ đề, nhóm cũng cung cấp liên kết chi tiết cho video hoạt ảnh tương ứng với các kịch bản trong đồ án:

E.7.1. Google Drive

- Bạn có thể nhấn vào **đây**
- Bạn cũng có thể truy cập đường link này: `https://drive.google.com/file/d/17m4i5Q4D-9nbcwIywxucs-0zaD2E1tPI/view?usp=sharing`
- Bạn cũng có thể truy cập vào mã QR này:

E.7.2. YouTube

- Bạn có thể nhấn vào **đây**
- Bạn cũng có thể truy cập đường link này: `https://youtu.be/YLofdJgss1k`
- Bạn cũng có thể truy cập vào mã QR này:

6 Part 3: Giải hệ phương trình và Phân tích hiệu năng

A Lý thuyết sai số và Tính ổn định số

A.1 Số điều kiện (Condition Number)

Định nghĩa A.1.1 (Số điều kiện). *Số điều kiện* của ma trận $\mathbf{A}$ đối với chuẩn p là:

$$\kappa_p(\mathbf{A}) = \|\mathbf{A}\|_p \cdot \|\mathbf{A}^{-1}\|_p. \tag{1}$$

Đối với chuẩn spectral (chuẩn 2): $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$.

Định lý A.1.2 (Phân tích sai số nghiệm). *Nếu* $\mathbf{A}\mathbf{x} = \mathbf{b}$ *là nghiệm đúng và* $\mathbf{A}\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$ *là nghiệm tính với nhiễu dữ liệu, thì:*

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \cdot \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}. \tag{2}$$

Khi $\kappa(\mathbf{A})$ *lớn, hệ bị điều kiện kém (ill-conditioned): sai số nhỏ trong dữ liệu đầu vào gây ra sai số lớn trong nghiệm.*

A.2 So sánh các phương pháp giải hệ phương trình

Phương pháp	**Loại**	**Điều kiện áp dụng**	**Chi phí**	**Ổn định**
Gauss (Partial Pivot)	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$O(n^3)$	Cao
LU với pivot	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$O(n^3)$	Cao
QR (Householder)	Trực tiếp	Mọi $\mathbf{A}$ (kể cả chữ nhật)	$O(n^3)$	Rất cao
Cholesky	Trực tiếp	$\mathbf{A}$ đối xứng xác định dương	$O(n^3/6)$	Rất cao
Gauss–Seidel	Lặp	Chéo trội hàng / SPD	$O(kn^2)$	Trung bình

Bảng 1: So sánh các phương pháp giải hệ phương trình

A.3 Phương pháp lặp Gauss-Seidel

A.3.1. Cơ sở lý thuyết

Phân rã $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ (đường chéo $\mathbf{D}$, tam giác dưới thuần $\mathbf{L}$, tam giác trên thuần $\mathbf{U}$). Ta có công thức lặp:

$$\mathbf{x}^{(k+1)} = -(\mathbf{D}+\mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k)} + (\mathbf{D}+\mathbf{L})^{-1}\mathbf{b}. \tag{12}$$

Nếu ta viết theo từng thành phần thì công thức lặp sẽ có dạng như sau:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right), \quad i = 1,\ldots,n. \tag{13}$$

Điều kiện hội tụ: $\mathbf{A}$ chéo trội chặt hàng, tức $|a_{ii}| > \sum_{j\neq i}|a_{ij}|$ với mọi i.

A.3.2. Các bước thực hiện thuật toán

Thuật toán Gauss-Seidel được sử dụng để giải gần đúng hệ phương trình tuyến tính $Ax = b$. Dựa trên mã nguồn cài đặt, thuật toán trải qua các bước sau:

- **Bước 1: Kiểm tra tính hợp lệ của ma trận đầu vào**
  - Cần đảm bảo ma trận hệ số A là ma trận *chéo trội ngặt theo hàng*. Nghĩa là với mọi hàng i, trị tuyệt đối của phần tử trên đường chéo chính phải lớn hơn tổng trị tuyệt đối của các phần tử còn lại trên cùng hàng đó: $|A_{ii}| > \sum_{j\neq i}|A_{ij}|$. Nếu vi phạm, thuật toán dừng và báo lỗi.
  - Kiểm tra các phần tử trên đường chéo chính. Nếu tồn tại $A_{ii} = 0$, phép chia ở bước cập nhật nghiệm sẽ gây lỗi chia cho 0. Thuật toán dừng và báo lỗi.
- **Bước 2: Khởi tạo**
  - Khởi tạo vector nghiệm ban đầu $x^{(0)}$ với toàn các giá trị 0.
  - Xác định số lần lặp tối đa (`max_iter` = 1000) và sai số chấp nhận được (`tol` = 10^{-16}).
- **Bước 3: Quá trình lặp (Lặp k từ 1 đến `max_iter`)**
  - Lưu lại giá trị vector nghiệm hiện tại vào x_{old} để so sánh đánh giá sai số ở cuối bước.

- Lặp qua từng phương trình i (từ 1 đến n) để cập nhật nghiệm x_i theo công thức Gauss-Seidel. Khác với phương pháp Jacobi, Gauss-Seidel sử dụng ngay các nghiệm x_j vừa được tính toán ở cùng bước lặp hiện tại:

$$x_i^{(k+1)} = \frac{1}{A_{ii}} \left(b_i - \sum_{j=1}^{i-1} A_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} A_{ij} x_j^{(k)} \right) \tag{3}$$

- **Bước 4: Kiểm tra điều kiện hội tụ**
  - Sau khi cập nhật xong vector x cho bước lặp thứ k, tính sai số (chuẩn vô cùng) giữa nghiệm mới và nghiệm cũ: $error = ||x - x_{old}||_\infty = \max_i |x_i - x_{old_i}|$.
  - Nếu $error < tol$, thuật toán được xem là đã hội tụ. Dừng vòng lặp và trả về vector nghiệm x.
- **Bước 5: Cảnh báo chưa hội tụ**
  - Nếu vòng lặp chạy hết `max_iter` lần mà sai số vẫn chưa đạt yêu cầu, in ra thông báo cảnh báo thuật toán chưa hội tụ và trả về vector x của lần lặp cuối cùng.

A.3.3. Mã giả

Dưới đây là mã giả của thuật toán được mô tả qua môi trường `algorithm`:

Algorithm 15 Phương pháp Gauss-Seidel

Require: Ma trận vuông $A \in \mathbb{R}^{n \times n}$, vector $b \in \mathbb{R}^n$, số lần lặp tối đa max_iter, sai số tol
Ensure: Vector nghiệm x

```
▷ Kiểm tra tính hợp lệ
if Không Phải Ma Trận Chéo Trội Ngặt(A) then
    throw Lỗi "Ma trận không chéo trội ngặt hàng"
end if
if ∃i ∈ {1..n} : A_ii = 0 then
    throw Lỗi "Phần tử trên đường chéo bằng 0"
end if

▷ Khởi tạo
Khởi tạo vector x ← (0, 0, ..., 0)^T

▷ Vòng lặp tính toán
for k = 1 to max_iter do
    x_old ← x
    for i = 1 to n do
        s ← 0
        for j = 1 to n do
            if j ≠ i then
                s ← s + A_ij · x_j        ▷ x_j tự động lấy giá trị mới nhất nếu j < i
            end if
        end for
        x_i ← (b_i − s) / A_ii
    end for

▷ Đánh giá sai số hội tụ
    error ← max_i |x_i − x_old_i|
    if error < tol then
        return x
    end if
end for
print "Cảnh báo: Thuật toán chưa hội tụ"
return x
```

B Cài đặt và phân tích thực nghiệm

Mô tả kịch bản thực nghiệm: Thực nghiệm đo lường thời gian thực thi (trung bình 5 lần chạy) và sai số tương đối $L2$ của 4 phương pháp: Gauss, LU, QR và Gauss-Seidel. Dữ liệu thử nghiệm gồm 3 loại ma trận: Ngẫu nhiên bình thường, SPD (well-conditioned)

và Hilbert (ill-conditioned) với các kích thước $n \in \{50, 100, 200, 500, 1000\}$.

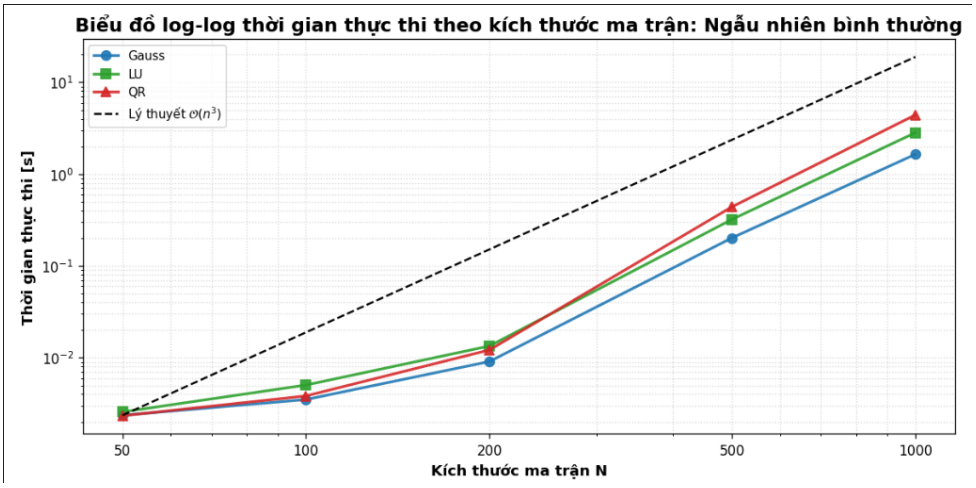
B.1 Ma trận ngẫu nhiên bình thường

B.1.1. Phân tích hiệu năng

==== Thời gian thực thi (giây) ====

solver / n	Gauss	LU	QR
50	0.00236112	0.00256702	0.00233346
100	0.00350322	0.0050402	0.00383474
200	0.00908746	0.0134075	0.0121466
500	0.201813	0.320678	0.439671
1000	1.6429	2.82555	4.41113

Bảng 2: Bảng số liệu thời gian thực thi của các phương pháp với ma trận ngẫu nhiên bình thường



Hình 1: Hiệu năng (thời gian thực thi) đối với ma trận ngẫu nhiên bình thường

Dựa trên biểu đồ Log-Log về thời gian thực thi:

- **Xu hướng đường biểu diễn:** Các đường thời gian của Gauss, LU và QR đều là những đường thẳng tăng đều, mượt và ổn định. Điều này cho thấy thuật toán hoạt động ổn định, không có hiện tượng gián đoạn hay tăng đột biến khi kích thước tăng từ $n = 50$ lên $n = 1000$.

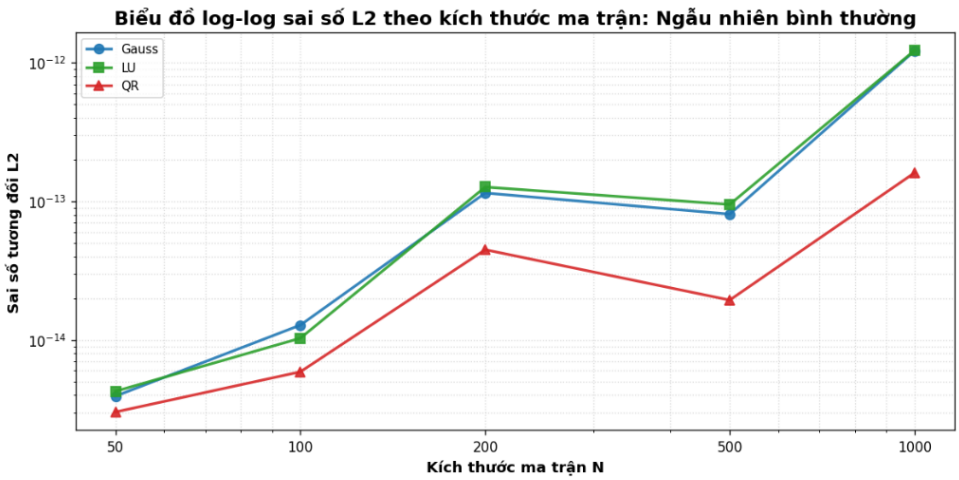
- **Đặc điểm từng thuật toán:**
  - **Gauss:** Nhanh nhất do số lượng phép tính tối thiểu
  - **LU & QR (Householder):** Chậm hơn Gauss nhưng vẫn giữ độ dốc ổn định, phản ánh chi phí phân rã ma trận
  - **Gauss $\geq$ LU $\geq$ QR**

B.1.2. Phân tích độ ổn định sai số

==== Sai số tương đối L2 ====

solver / n	Gauss	Gauss-Seidel	LU	QR
50	3.93e-15	Lỗi: Ma trận không chéo trội ngặt	4.26e-15	3.02e-15
100	1.28e-14	Lỗi: Ma trận không chéo trội ngặt	1.03e-14	5.89e-15
200	1.15e-13	Lỗi: Ma trận không chéo trội ngặt	1.27e-13	4.48e-14
500	8.09e-14	Lỗi: Ma trận không chéo trội ngặt	9.50e-14	1.94e-14
1000	1.22e-12	Lỗi: Ma trận không chéo trội ngặt	1.23e-12	1.61e-13

Bảng 3: Bảng số liệu sai số L2 của các phương pháp với ma trận ngẫu nhiên



Hình 2: Độ ổn định (sai số $L2$) đối với ma trận ngẫu nhiên bình thường

Sai số được đánh giá qua:

$$\frac{\|A\hat{x} - b\|_2}{\|b\|_2}.$$

- **Xu hướng đường biểu diễn:**

- **Giai đoạn đầu** ($N < 200$)**:** Cả ba phương pháp đều có sai số rất nhỏ (xấp xỉ từ 3×10^{-15} đến 5×10^{-15}). Khi N tăng đến khoảng 100, sai số của QR tăng nhẹ (vẫn dưới 10^{-14}), trong khi Gauss và LU tăng nhanh hơn và vượt ngưỡng này.
- **Giai đoạn sau** ($200 \leq N \leq 1000$)**:** Sai số tăng mạnh khi $N = 200$, sau đó giảm nhẹ tại $N = 500$ rồi tiếp tục tăng. Ở $N = 1000$, Gauss và LU tăng vọt, vượt 10^{-12}..

- **So sánh các phương pháp:**
  - **QR (Householder):** Luôn cho sai số thấp nhất. Tại $n = 1000$:

$$\approx 1.61 \times 10^{-13}.$$

  - **Gauss & LU:** Sai số cao hơn, khoảng:

$$\approx 1.2 \times 10^{-12}.$$

Nhìn chung, khi kích thước ma trận tăng, tất cả các phương pháp đều kém chính xác hơn; tuy nhiên, QR (Householder) là lựa chọn tốt nhất nếu cần giảm sai số trong tính toán với ma trận lớn.

B.1.3. Khó khăn khi kiểm thử phương pháp lặp Gauss–Seidel

- **Kết quả thực nghiệm:** Với ma trận ngẫu nhiên, thuật toán trả về lỗi do không thỏa mãn điều kiện hội tụ.
- **Lý thuyết:** Phương pháp Gauss–Seidel yêu cầu điều kiện chéo trội:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \ldots, n.$$

- **Hệ quả:** Ma trận ngẫu nhiên thường không đảm bảo cấu trúc chéo trội, do đó không thể so sánh trực tiếp với các phương pháp trực tiếp trong trường hợp này.

B.1.4. Kết luận

- **Về hiệu năng:** Các thuật toán hoạt động ổn định, thời gian thực thi tăng đều và phù hợp với lý thuyết.

- **Về độ ổn định:**
  - **QR:** ổn định nhất với sai số nhỏ nhất.
  - **Gauss, LU:** ổn định nhưng kém hơn QR khi kích thước ma trận tăng.

B.2 Ma trận SPD (Well-conditioned)

B.2.1. Phân tích hiệu năng

==== Thời gian thực thi (giây) ====

solver / n	Gauss	Gauss-Seidel	LU	QR
50	0.0008907	0.00266082	0.00152618	0.00138664
100	0.00206738	0.00266664	0.0018935	0.0034182
200	0.0074134	0.00505666	0.00819158	0.0111253
500	0.169922	0.0167001	0.285766	0.411006
1000	1.58247	0.0407328	2.65735	4.35351

Bảng 4: Bảng số liệu thời gian thực thi các phương pháp với ma trận SPD

Hình 3: Hiệu năng (thời gian thực thi) đối với ma trận SPD

Dựa trên biểu đồ thời gian, ta thấy sự phân hóa rõ rệt giữa các nhóm thuật toán:

- **Nhóm phương pháp trực tiếp (Gauss, LU, QR):**
  - **Đặc điểm thực nghiệm (tại $n = 1000$):** Gauss ($1.58\,s$) nhanh nhất, tiếp theo là LU ($2.65\,s$) và cuối cùng là QR ($4.35\,s$).

– **Giải thích:** Sự chênh lệch đến từ số lượng phép tính dấu phẩy động khác nhau giữa khử dòng và các phép biến đổi trực giao.

- **Phương pháp lặp Gauss–Seidel:**

  – **Xu hướng:** Có độ dốc thấp hơn cho thấy thời gian thực thi nhanh hơn và tiệm cận độ phức tạp

  $$O(k \cdot n^2),$$

  với k là số bước lặp.

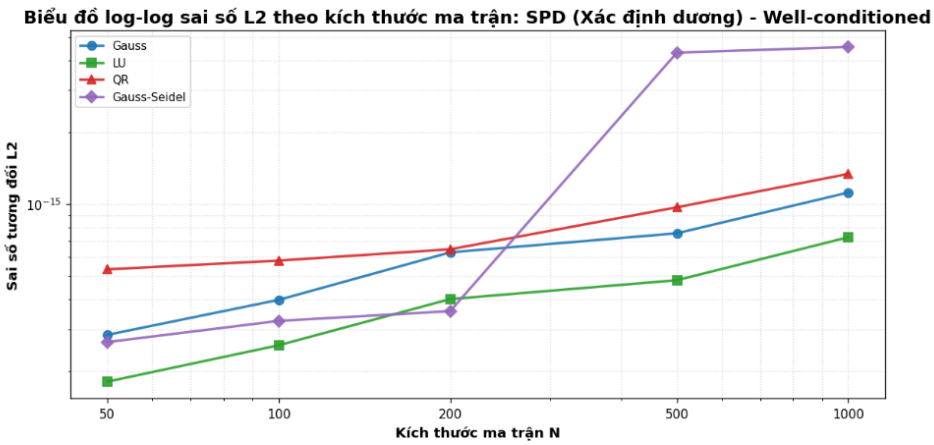
  – **Thực nghiệm:** Tại $n = 1000$, thời gian của phương pháp Gauss-Seidel chỉ khoảng $0.04\,s$, cho thấy hiệu năng vượt trội so với các phương pháp trực tiếp.

B.2.2. Phân tích độ ổn định sai số

==== Sai số tương đối L2 ====

solver / n	Gauss	Gauss-Seidel	LU	QR
50	2.84e-16	2.65e-16	1.81e-16	5.34e-16
100	3.98e-16	3.25e-16	2.57e-16	5.81e-16
200	6.29e-16	3.57e-16	4.01e-16	6.48e-16
500	7.56e-16	4.31e-15	4.81e-16	9.71e-16
1000	1.12e-15	4.55e-15	7.28e-16	1.34e-15

Bảng 5: Bảng số liệu sai số $L2$ các phương pháp với ma trận SPD



Hình 4: Độ ổn định đối với ma trận SPD

Độ ổn định được đánh giá qua sai số:

$$\frac{\|A\hat{x} - b\|_2}{\|b\|_2}.$$

- **Số điều kiện (Condition Number):** Với ma trận SPD (well-conditioned), số điều kiện nhỏ:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

  Do đó sai số được kiểm soát tốt theo bất đẳng thức:

$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \kappa(A) \cdot \frac{\|\delta b\|}{\|b\|}.$$

- **Sự ổn định của các phương pháp:**
  - **LU:** Có sai số thấp nhất (xấp xỉ 10^{-15} đến 10^{-14}) và ổn định nhất trong toàn bộ quá trình.
  - **Gauss và QR:** Sai số tăng đều; QR ban đầu cao hơn nhưng dần tiệm cận Gauss khi n tăng.
  - **Gauss–Seidel:** Có xu hướng mất ổn định khi N lớn và sai số tăng mạnh. Sai số có thể xuất hiện "gấp khúc" tại $n = 200$, có thể giải thích bởi sự tích lũy sai số làm tròn qua nhiều phép tính thành phần trong mỗi bước lặp:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right).$$

B.2.3. Lợi thế của phương pháp lặp Gauss–Seidel trên ma trận SPD

Ma trận SPD mang lại nhiều lợi thế lý thuyết:

- **Hội tụ chắc chắn:** Đảm bảo hội tụ ngay cả khi không chéo trội ngặt.
- **Tốc độ hội tụ nhanh:** Khi số điều kiện nhỏ, số bước lặp k cần thiết cũng nhỏ, do đó thời gian có thể giảm đáng kể so với độ phức tạp $O(n^3)$ của các phương pháp trực tiếp.

B.2.4. Kết luận

- **Về hiệu năng:** Gauss–Seidel là lựa chọn tối ưu về tốc độ cho ma trận SPD kích thước lớn.
- **Về độ ổn định:** Nhóm phương pháp trực tiếp (đặc biệt LU/QR/Gauss) cho độ chính xác cao và ổn định.
- **Khả năng mở rộng:** Kết hợp giữa tốc độ của Gauss–Seidel và độ chính xác của LU là phương án hợp lý cho các hệ thực tế.

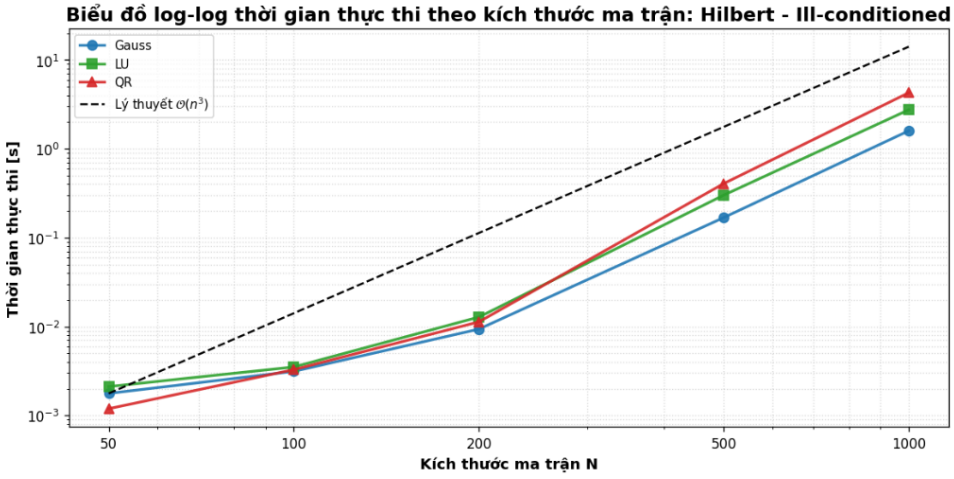
B.3 Ma trận Hilbert (Ill-conditioned)

B.3.1. Phân tích hiệu năng

==== Thời gian thực thi (giây) ====

solver / n	Gauss	LU	QR
50	0.00176742	0.00211118	0.0011903
100	0.00313214	0.00350982	0.00327462
200	0.00936698	0.0128014	0.0112973
500	0.168404	0.301232	0.406683
1000	1.59872	2.7554	4.28874

Bảng 6: Bảng số liệu thời gian thực thi các phương pháp với ma trận Hilbert



Hình 5: Hiệu năng đối với ma trận Hilbert

Dựa trên biểu đồ thời gian và dữ liệu thực tế:

- **Xu hướng đường biểu diễn:** Trái ngược với sự "hỗn loạn" của sai số, các đường thời gian của Gauss, LU và QR vẫn là những đường thẳng mượt trên thang Log–Log.
- **Tính ổn định về hiệu năng:** Chi phí tính toán chủ yếu phụ thuộc vào kích thước ma trận n và không phụ thuộc đáng kể vào giá trị phần tử:

$$O(n^3).$$

- **Đặc điểm từng thuật toán (tại $n = 1000$):**
  - **Gauss** ($1.59\,s$)**:** Nhanh nhất.
  - **LU** ($2.75\,s$)**:** Chậm hơn Gauss khoảng 1.7 lần.
  - **QR (Householder)** ($4.28\,s$)**:** Chậm nhất do chi phí biến đổi trực giao.

B.3.2. Phân tích độ ổn định sai số

==== Sai số tương đối L2 ====

solver / n	Gauss	Gauss-Seidel	LU	QR
50	1.04e-15	Lỗi: Ma trận không chéo trội ngặt	5.09e-16	1.05e-15
100	2.43e-14	Lỗi: Ma trận không chéo trội ngặt	2.78e-14	9.79e-15
200	3.68e-15	Lỗi: Ma trận không chéo trội ngặt	3.85e-15	9.01e-14
500	5.34e-14	Lỗi: Ma trận không chéo trội ngặt	2.72e-14	3.50e-14
1000	1.43e-14	Lỗi: Ma trận không chéo trội ngặt	1.66e-14	1.68e-13

Bảng 7: Bảng số liệu sai số $L2$ các phương pháp với ma trận Hilbert

Hình 6: Độ ổn định đối với ma trận Hilbert

Đây là nơi bản chất ill-conditioned của ma trận Hilbert thể hiện rõ nhất:

- **Xu hướng "gấp khúc" mạnh:** Đường sai số dao động liên tục, trồi sụt thất thường; không còn mượt hoặc tăng tuyến tính.
- **Giải thích qua số điều kiện:**

$$\kappa_p(A) = \|A\|_p \cdot \|A^{-1}\|_p.$$

  Với Hilbert, $\kappa(A)$ tăng rất nhanh theo n, khiến hệ cực kỳ nhạy với sai số.
- **Hệ quả lên sai số nghiệm:**

$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \kappa(A) \cdot \frac{\|\delta b\|}{\|b\|}.$$

  Vì vậy, sai số làm tròn nhỏ cũng có thể bị khuếch đại mạnh, làm kết quả dao động khó kiểm soát.
- **Biểu hiện trên đồ thị:**
  - Tại $n = 200$, sai số QR tăng vọt gần 10^{-13}.
  - Nhưng tại $n = 500$ sai số lại thấp hơn.

  Điều này cho thấy đường sai số bị **gấp khúc mạnh** và thiếu ổn định.
- **Kết luận:** Khi n lớn, các phương pháp trực tiếp **không còn giữ được độ ổn định số học** khi áp dụng cho ma trận Hilbert.

B.3.3. Sự bất khả thi của phương pháp lặp Gauss–Seidel

- **Kết quả thực nghiệm:** Gauss–Seidel trả về giá trị `null` do không hội tụ.
- **Nguyên nhân lý thuyết:** Phương pháp yêu cầu điều kiện hội tụ (chéo trội):

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|.$$

  Tuy nhiên, ma trận Hilbert **không thỏa mãn chéo trội**, do đó thuật toán có thể phân kỳ ngay từ đầu.

- **Hệ quả:**
  - Không thể áp dụng phương pháp lặp.
  - Không thể so sánh trực tiếp với các phương pháp trực tiếp trong trường hợp này.

B.3.4. Kết luận

- **Về hiệu năng:** Các thuật toán vẫn tuân theo độ phức tạp $O(n^3)$ do đó đường thời gian tăng đều, mượt và ổn định.
- **Về độ ổn định:** Đường sai số dao động mạnh và liên tục "gấp khúc", thể hiện tính **không ổn định nghiêm trọng** của bài toán khi ma trận ill-conditioned.

C Tổng kết

Qua quá trình thực nghiệm, nhóm rút ra kết luận chung:

- **Về thời gian thực thi:** Các phương pháp trực tiếp (Gauss, LU, QR) đều tuân thủ chặt chẽ độ phức tạp lý thuyết $O(n^3)$. Phương pháp lặp Gauss-Seidel cho thấy sức mạnh vượt trội về tốc độ $O(k \cdot n^2)$ khi xử lý các ma trận lớn thỏa mãn điều kiện chéo trội hoặc SPD.
- **Về độ ổn định số học:** Với các ma trận có số điều kiện nhỏ (ngẫu nhiên, SPD), các phương pháp đều cho sai số rất thấp ($< 10^{-14}$). Tuy nhiên, khi đối mặt với ma trận ill-conditioned (Hilbert), các phương pháp trực tiếp đều bộc lộ sự mất ổn định do khuếch đại sai số làm tròn, trong khi phương pháp lặp hoàn toàn phân kỳ.

- **Khuyến nghị áp dụng:** Nên sử dụng Gauss/LU/QR cho các hệ phương trình tổng quát cỡ vừa và nhỏ. Đối với các hệ thưa, kích thước cực lớn hoặc mang tính chất SPD (thường gặp trong học máy, mô phỏng vật lý), phương pháp lặp Gauss-Seidel là sự lựa chọn tối ưu.

7 KẾT LUẬN

A Tóm tắt kết quả đạt được

Trải qua quá trình nghiên cứu và thực hiện đồ án, nhóm đã hoàn thành đầy đủ các mục tiêu đề ra với các kết quả cụ thể như sau:

- **Về thuật toán:** Cài đặt thành công "từ đầu" (from scratch) phép khử Gauss với kỹ thuật chọn phần tử chốt (*Partial Pivoting*), từ đó xây dựng hệ thống các hàm tính toán nền tảng bao gồm: giải hệ phương trình tuyến tính, tính định thức, tìm ma trận nghịch đảo và xác định hạng/cơ sở của các không gian ma trận.
- **Về phân tích hiệu năng:** Thực hiện so sánh thực nghiệm chi tiết giữa các phương pháp trực tiếp (Gauss, LU, QR) và phương pháp lặp (Gauss-Seidel) trên nhiều loại ma trận khác nhau (Ngẫu nhiên, SPD, Hilbert). Kết quả cho thấy sự phù hợp giữa thực nghiệm và lý thuyết về độ phức tạp $O(n^3)$ cũng như tính ổn định số học.
- **Về trực quan hóa:** Xây dựng các hoạt ảnh Manim sinh động, mô phỏng rõ nét bản chất của phép phân rã ma trận và chéo hóa, giúp chuyển đổi các khái niệm trừu tượng thành hình ảnh dễ hiểu.
- **Về kiểm chứng:** Tất cả các hàm tự cài đặt đều đã được đối chiếu và cho kết quả khớp với các thư viện chuẩn như NumPy và SciPy.

B Bài học rút ra

- **Tầm quan trọng của tính ổn định số:** Nhóm hiểu rõ rằng trong tính toán khoa học, một thuật toán đúng về mặt toán học là chưa đủ, mà còn cần phải ổn định về mặt số học. Kỹ thuật *Partial Pivoting* là minh chứng rõ nhất cho việc kiểm soát sai số làm tròn trên máy tính.

- **Kỹ năng lập trình và tối ưu:** Việc cài đặt các thuật toán ma trận giúp nhóm nâng cao kỹ năng xử lý mảng, tối ưu hóa vòng lặp và làm quen với việc quản lý độ phức tạp thuật toán.
- **Tư duy phân tích thực nghiệm:** Qua việc vẽ đồ thị log-log và phân tích ma trận Hilbert, nhóm đã học được cách đánh giá một thuật toán không chỉ qua kết quả đúng/sai mà còn qua hiệu suất xử lý dữ liệu lớn và khả năng chịu lỗi trước dữ liệu "nhiễu".

C Khó khăn gặp phải

- **Xử lý các trường hợp đặc biệt:** Việc biện luận nghiệm cho hệ phương trình vô số nghiệm và trích xuất cơ sở cho không gian nghiệm đòi hỏi sự tỉ mỉ trong việc quản lý các biến tự do và biến phụ thuộc.
- **Vấn đề hội tụ của phương pháp lặp:** Khi thực nghiệm phương pháp Gauss-Seidel, nhóm gặp khó khăn khi thuật toán không hội tụ trên các ma trận không chéo trội hoặc có số điều kiện lớn như ma trận Hilbert.
- **Công cụ trực quan hóa:** Việc làm quen với thư viện Manim đòi hỏi nhiều thời gian để tinh chỉnh thời gian (timing) và bố cục hoạt ảnh sao cho vừa đảm bảo tính thẩm mỹ, vừa giữ được tính chính xác của các bước biến đổi ma trận.

8 Tài liệu tham khảo

Tài liệu

[1] Gilbert Strang. *Introduction to Linear Algebra, Sixth Edition (2023)*.
https://math.mit.edu/~gs/linearalgebra/ila6/indexila6.html

[2] Lloyd N. Trefethen & David Bau III. *Numerical Linear Algebra*.
https://www.stat.uchicago.edu/~lekheng/courses/309/books/Trefethen-Bau.pdf

[3] Manim Community Developers. *Manim-Mathematical Animation Framework, v0.18*.
https://docs.manim.community/

[4] 3Blue1Brown. *Essence of Linear Algebra*.
https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab

[5] Jake VanderPlas. *Python Data Science Handbook*.
https://jakevdp.github.io/PythonDataScienceHandbook/